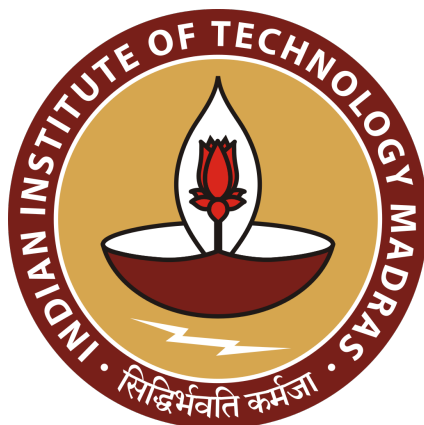


DEPARTMENT OF COMPUTER SCIENCE &
ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

Dynamic Louvain on GPUs with CUDA



A Report

Submitted by

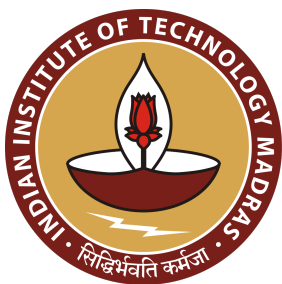
CHANDALURU HEMA VENKATA RAADHES

For the award of the degree

Of

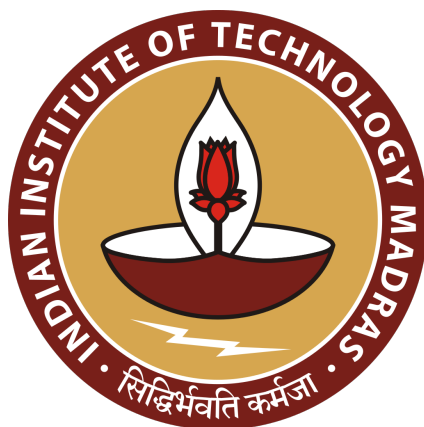
BACHELOR OF TECHNOLOGY

June 2026



DEPARTMENT OF COMPUTER SCIENCE &
ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

Dynamic Louvain on GPUs with CUDA



A Report

Submitted by

CHANDALURU HEMA VENKATA RAADHES

For the award of the degree

Of

BACHELOR OF TECHNOLOGY

June 2026

*I have no special talent. I am only
passionately curious.*

– Albert Einstein

I dedicate my report to my parents who shaped my educational journey since childhood.

To my professors and guide whose teachings and guidance has been core to my university education.

To my friends whose energy, support and time have been great gifts which made my university life memorable, fun and meaningful.

CERTIFICATE

This is to undertake that the Report titled **Dynamic Louvain on GPUs with CUDA**, submitted by me to the Indian Institute of Technology Madras, for the award of **Bachelor of Technology**, is a bona fide record of the research work done by me under the supervision of **Dr. Rupesh Nasre**. The contents of this Report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Chennai 600036

Chandaluru Hema Venkata Raadhes

Date: June 2026

Dr. Rupesh Nasre

Research advisor

Professor

Department of Computer Science and Engineering

IIT Madras

ACKNOWLEDGEMENTS

I would like to acknowledge my guide Professor Rupesh Nasre for agreeing to be my guide for my BTP and for his valuable mentorship during my project.

I also thank Professor Aishwarya Thiruvengadam for coordinating the BTP for this semester and my faculty advisor - Professor Yadu Vasudev for his guidance throughout my degree.

I also acknowledge the use of LLMs (Gemini and Claude) for assistance in streamlining research by *summarizing content*, providing research suggestions, generating scripts, assisting in *table & pgfplot generation from benchmarking statistics*, enhancing english in the report and other latex based assistance. Any AI-content has either been rigorously reviewed for correctness or processed by me and handwritten.

ABSTRACT

Community detection on graphs is a core task in social, biological, and information network analysis. Community detection is the problem of partitioning a graph's vertices into groups (called communities) that are densely connected within each group but only sparsely connected between groups. Intuitively, these communities correspond to meaningful structure in the underlying system - friend circles/societies/universities in a social network, functionally related proteins in a biological network, or topically related pages on the web. Hence it is a core task in social, biological and information network analysis among other fields.

These real-world networks are always changing (new connections on social media websites, new citations etc.) which motivates the development of algorithms that maintain community structure as the graph evolves rather than recomputing it from scratch. In this project we address this with the implementation of a dynamic Louvain community detection algorithm for GPUs that efficiently maintains the community structure under graph updates (edge additions/deletions).

State of the art CPU implementations for dynamic Louvain (delta screening, DF-Louvain) achieve strong performance through selective processing of vertices and edges, however GPU implementations remain largely static. Well known libraries like NVIDIA cuGraph and other GPU based Louvain community detection implementations like cuVite and Nido support only a static Louvain algorithm on the GPU.

This work presents a static Louvain implementation for GPUs together with three dynamic variants - Naive-Dynamic, Delta-Screening, and Dynamic-Frontier - adapted from their CPU formulations to CUDA. To our best knowledge this constitutes one of the only publicly-available dynamic Louvain community detection implementation targeting GPUs apart from [Muthavarapu](#). Utilizing the NVIDIA CUDA architecture,

key algorithmic phases such as modularity-gain calculation and node movement are parallelized to maximize throughput and exploit the GPU’s parallelism.

In this report we detail the background, literature review done, design considerations, benchmarking methodology and results of the implemented algorithms.

We evaluate on real world graphs achieving impressive performance gains with the GPU static Louvain implementation of $1.74\times$ and $2.18\times$ compared to cuGraph and NetworKit respectively on the six largest SNAP graphs used for benchmarking (geometric average). Notably, we achieve a speedup of $4\times$, $15\times$ and $450\times$ compared to cuGraph, NetworKit, NetworkX respectively on web-Google graph of over 4 million edges however we observe a drop in speed on two graphs of approximately 30 million and 110 million edges likely due to graph structure.

For the dynamic setting, the three variants update communities by reprocessing only a subset of the vertices based on the batch updates, starting from the previous partition rather than re-clustering from scratch. This makes each per-batch update much faster than every static baseline, all of which would need to re-cluster the entire graph after each batch: for 4% dynamic batch updates on the SNAP temporal graphs the DF-variant is $20.14\times$ faster than cuGraph and $5.58\times$ faster than NetworKit (geometric average), and hundreds of times faster than the single-threaded NetworkX. Crucially, this speed costs little to no quality.

CONTENTS

	Page
ACKNOWLEDGEMENTS	i
ABSTRACT	iii
LIST OF TABLES	ix
LIST OF FIGURES	xi
CHAPTER 1 INTRODUCTION	1
1.1 Motivation	1
1.2 Research Objectives	2
1.3 Contributions	2
CHAPTER 2 PRELIMINARIES AND BACKGROUND	5
2.1 Modularity	5
2.1.1 Modularity Gain	6
2.1.2 Derivation of modularity gain	7
2.2 Louvain Community Detection Algorithm	8
2.2.1 Phase 1 : Modularity Optimization	8
2.2.2 Phase 2: Community Aggregation	9
2.2.3 A Worked Example	9
2.3 NVIDIA GPUs	13
2.3.1 Philosophy	13
2.3.2 Architecture	14
2.4 CUDA Programming Model	16
2.4.1 Launching A Kernel	17
2.4.2 Memory	18
2.4.3 Another kernel	19
2.4.4 Functions	20
2.4.5 Thrust: A library	21
2.4.6 Synchronisation Primitives	24
2.4.7 Error Checking	26
2.4.8 The NVCC compiler	26
CHAPTER 3 LITERATURE REVIEW	29
3.1 Modularity	29
3.2 The Original Paper	31
3.3 CPU-Based Static Louvain Algorithms	32
3.3.1 A heuristics deep dive	32
3.3.2 Multicore CPU Implementations	33

3.3.3	Vite	37
3.3.4	Leiden	38
3.4	GPU-Based and Hybrid Static Louvain Algorithm	39
3.4.1	The first GPU accelerated Louvain	39
3.4.2	HyDetect	40
3.4.3	ACLM	40
3.4.4	Nido	41
3.4.5	cuVite	41
3.4.6	cuGraph	42
3.4.7	ν -Louvain	42
3.5	CPU-Based Dynamic Louvain Algorithm	43
3.5.1	Shang <i>et al.</i> (2014)	43
3.5.2	Cordeiro <i>et al.</i> (2016)	43
3.5.3	C-Blondel	44
3.5.4	Naive-Dynamic, Delta Screening & DF-Louvain	45
3.6	GPU-Based Dynamic Louvain Algorithm	45
CHAPTER 4 WORK DETAIL		47
4.1	UGRC Work	47
4.2	Project Timeline	48
4.3	Dynamic Louvain Community Detection Algorithms	49
4.3.1	Dynamic Graphs and Batch Updates	49
4.3.2	Naive Dynamic	50
4.3.3	Delta Screening	50
4.3.4	Dynamic Frontier	51
4.4	Implementation	51
4.4.1	Data structures	51
4.4.2	The local-moving kernel	53
4.4.3	Aggregation	54
4.4.4	The Louvain Driver	55
4.5	Marking and the dynamic variants	57
4.6	Synchronisation Considerations	58
4.7	Other Considerations	60
4.7.1	Vertex-based versus edge-based parallelism	60
4.7.2	Cooperative kernels and the grid-size limit	60
4.7.3	Convergence thresholds and the move epsilon	61
4.7.4	A modularity-gain bug from self-loops	62
4.7.5	Best-target moves instead of any improving move	63
4.7.6	Coalesced memory access	63
CHAPTER 5 BENCHMARKING & RESULTS		65
5.1	Graphs	65
5.1.1	Classic Benchmark Graphs	65
5.1.2	SNAP Graphs	66
5.1.3	SNAP Temporal Graphs	67

5.2	Consistent Graph Format	68
5.2.1	Static SNAP Graphs Format & Processing	69
5.2.2	Temporal SNAP Graphs Format	70
5.3	Baseline Algorithms	71
5.3.1	NetworkX	72
5.3.2	NetworKit	72
5.3.3	cuGraph	73
5.4	Environment	73
5.5	Static Louvain Benchmarking	74
5.5.1	Classic Graph Sanity Check	74
5.5.2	Performance on SNAP Graphs	76
5.5.3	Performance on large SNAP Graphs	79
5.5.4	Graph Structure Analysis	81
5.6	Dynamic Louvain Benchmarking	83
5.6.1	SNAP Temporal Experiment (80% static, $5 \times 4\%$ batches) . .	83
5.6.2	Quality Degradation Experiments	92
CHAPTER 6	CONCLUSION	95
6.1	Future Work	95
6.1.1	Algorithmic Improvements	95
6.1.2	Evaluation Improvements	96
6.1.3	Publication	97
APPENDIX A	GENERAL NOTES	99
A.1	The Local-Moving Kernel	99
REFERENCES		103

LIST OF TABLES

Table	Caption	Page
2.1	CUDA function execution space specifiers	21
4.1	Project timeline.	48
5.1	Static SNAP benchmark graphs.	66
5.2	SNAP temporal benchmark graphs Leskovec and Krevl (2014). “Temporal edges” is the full timestamped stream; “static edges” is the de-duplicated graph.	67
5.3	Candidate Louvain baselines.	71
5.4	Experimental Environment and Software Versions	74
5.5	Performance on the classic benchmark graphs.	75
5.6	Per-node comparison on Zachary’s karate club.	75
5.7	SNAP benchmark graphs in increasing edge-count order. Note graph vertex and edge count are listed post-processing using the converter mentioned in 5.2	77
5.8	Experimental Environment and Software Versions	79
5.9	Static Louvain on the large SNAP graphs (executed on the NVIDIA A100).	80
5.10	Degree statistics of the converted (pre-processed) benchmark graphs together with the runtime speedup ratio of our method over the cuGraph and NetworKit baselines (baseline runtime ÷ our runtime; a value below 1 means we are slower).	81
5.11	Modularity per batch on CollegeMsg.	84
5.12	Communities per batch on CollegeMsg.	84
5.13	Time (s) per batch on CollegeMsg.	85
5.14	Modularity per batch on sx-mathoverflow.	86
5.15	Communities per batch on sx-mathoverflow.	86
5.16	Time (s) per batch on sx-mathoverflow.	87
5.17	Modularity per batch on sx-askubuntu.	88
5.18	Communities per batch on sx-askubuntu.	88
5.19	Time (s) per batch on sx-askubuntu.	89
5.20	Modularity per batch on sx-superuser.	90
5.21	Communities per batch on sx-superuser.	90
5.22	Time (s) per batch on sx-superuser.	91
5.23	Average per-batch update time (milliseconds): the mean over the five insertion batches, excluding the initial static clustering.	92
5.24	Modularity per batch for the larger batch sizes (5%, 10%, 20% of $ E $)	94

LIST OF FIGURES

Figure	Caption	Page
2.1	A 9-node, 12-edge graph.	10
2.2	After the local-moving phase.	12
2.3	After aggregation.	12
2.4	GPU Device Architecture	14
2.5	Streaming Multiprocessor Architecture	15
3.1	Adapted from Brandes <i>et al.</i> (2008), © 2008 IEEE.	30
4.1	adapted from Nasre	63
5.1	The example batch applied to the initial graph.	69
5.2	Per-graph wall-clock time (seconds) of GPU Static Louvain, cuGraph, and NetworKit on the SNAP benchmark graphs. Each panel has an independent y-axis. NetworkX is omitted, as its runtimes would dominate the scale.	78
5.3	Modularity per batch on com-DBLP for a batch size of 2% of $ E $. The ND and DS variants pretty much coincide.	93

CHAPTER 1

INTRODUCTION

1.1 MOTIVATION

Graphs underpin many modern systems: from social media platforms, biological interaction networks, financial transaction ledgers, road networks to wikipedia pages. These are all naturally represented as graphs where the entities are vertices and the relationships are edges. Community detection is a fundamental analytic task on these graphs. This partitions the vertices into groups which are densely connected internally but sparsely connected to vertices outside their assigned group. Efficient and accurate community detection supports business tasks such as influence maximization, protein function prediction, and fraud detection among others.

The Louvain community detection method is described in [Blondel *et al.* \(2008\)](#). This algorithm is renowned for its speed and ability to optimize the modularity of the community partition. Modularity will be discussed further on in the thesis [2.1](#).

However, the Louvain algorithm was conceived for running on one graph snapshot. It processes the graph and produces a community partition. Real world graphs are dynamic - they constantly change. Vertices and edges arrive and disappear constantly. Some examples include:

- Construction of new roads in a city.
- Teenagers creating new Instagram accounts. Existing account holders adding new friends. Troublemakers having their accounts banned.
- New papers in a citation database.
- Updation of protein interaction findings.

Directly applying the Louvain method to dynamic real world graphs requires re-running

the whole algorithm from scratch when updates occur. This approach becomes computationally infeasible as the graph size becomes very large or updates become more frequent. Latency issues and unnecessary computation become prevalent in these scenarios. Dynamic variants of the Louvain algorithm aim to incrementally update the community partition as updates arrive in a practical manner (computationally and latency wise). This avoids a re-run of the original Louvain algorithm on the new updated graph.

1.2 RESEARCH OBJECTIVES

This project is a continuation of the UGRC (undergraduate research) work done under Dr. Rupesh Nasre by Raadhesh Chandalur. The UGRC work presented a CUDA based static Louvain algorithm and review on the dynamic Louvain algorithm.

- Understand the Louvain community detection algorithm.
- Research various improvements and modifications of the algorithm.
- Research prior work done on Louvain community detection algorithms directed at CPU and GPU computation. This work would involve knowing more about specific algorithmic improvements, optimizations for parallel computation and GPU optimizations that may be done for the algorithm.
- Expand upon the B.Tech Project work of **Muthavarapu**.
- Improve the static Louvain community detection algorithm implemented in the UGRC work and fix correctness issues as well as bugs in the code.
- Implement the CUDA based dynamic Louvain algorithm for GPUs.
- Profile, Optimize and benchmark the performance of the implementation against real world graphs.

1.3 CONTRIBUTIONS

- An extensive literature review of over **20 papers** spanning: CPU and GPU based implementations of the both static and dynamic Louvain community detection algorithms.

- Successfully setup and ran multiple baselines to possibly be used for comparison including **cuGraph**, **Nido**, **Vite**, **NetworkX** and **NetworKit**.
- GPU based static Louvain re-implemented after resolving bugs and correctness issues present in my UGRC implementation.
- An implementation of the **dynamic Louvain algorithm for GPUs** in three variants - Naive Dynamic, Delta Screening, Dynamic Frontier adapted from their CPU formulations in the literature.
- Extensive benchmarking of the implemented GPU Static Louvain and Dynamic Louvain variants (Naive Dynamic, Delta Screening and Dynamic Frontier).
 - Comparison against the Louvain algorithm from cuGraph, NetworkX and NetworKit libraries.
 - Survey, selection and pre-processing (into a common format) of real-world graphs from the SNAP dataset [Leskovec and Krevl \(2014\)](#).
 - Setup reusable scripts for quick and uniform benchmarking.
- The GPU static Louvain achieves a (geometric average) speed up of 1.74× and 2.18× compared with cuGraph and NetworKit over the six largest SNAP graphs. On one of the largest graphs (web-Google) we achieve 4×, 15× **and** 450× performance gains compared to **cuGraph**, **NetworKit**, **NetworkX** respectively.
- On the SNAP temporal graphs a dynamic batch update is **several times faster** than the GPU **cuGraph** and parallel **NetworKit** baselines, and **hundreds of times** faster than the single-threaded **NetworkX**. Crucially, this speed costs **little to no quality**.

The code and testing scripts along with the graphs (apart from LargeSnap's graphs which are too large to push to a repository) can be found in the GitHub repository [here](#).

CHAPTER 2

PRELIMINARIES AND BACKGROUND

2.1 MODULARITY

Modularity introduced by **Newman and Girvan (2004)** is a scoring metric which is used to evaluate the quality of communities formed by community detection algorithms. It is calculated as the difference between the fraction of edges within communities and the expected fraction of edges if the edges were distributed randomly. Higher modularity corresponds to a better community partition. The range of modularity is $[-0.5, 1]$.

For a weighted, undirected graph modularity is given by:

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \quad (2.1)$$

Where the terms are explained below:

- m : The sum of all weights of edges in the graph.
- i, j : Indices representing two nodes in the graph. The summation $\sum_{i,j}$ covers all possible pairs of nodes.
- A_{ij} : This is the weight of the edge between nodes i and j .
- k_i : This is weighted degree of node i (sum of weights of edges incident to node i).
- $\delta(c_i, c_j)$: This is the Kronecker delta function. It is defined as:

$$\delta(c_i, c_j) = \begin{cases} 1 & \text{when node } i \text{ and node } j \text{ are in the same community } c \\ 0 & \text{otherwise} \end{cases}$$

It is often useful to consider the contribution of each community to the total modularity. The above formula for entire graph modularity can be simplified to the following expression for the modularity per community.

$$Q_c = \frac{\Sigma_{in}(c)}{2m} - \left(\frac{\Sigma_{deg}(c)}{2m} \right)^2 \quad (2.2)$$

The terms are defined as follows:

- Q_c is the modularity contribution of community c .
- $\Sigma_{in}(c) = \sum_{i \in c, j \in c} w_{ij}$ is the sum of the weights of all edges within community c with each edge being considered twice.
- $\Sigma_{deg}(c) = \sum_{i \in c} k_i$ is the sum of the degrees (weighted, if applicable) of all vertices belonging to community c .
- m is the total sum of the weights of all edges in the graph.

The first term, $\frac{\Sigma_{in}(c)}{2m}$, is the observed fraction of all edge weight that falls inside community c . The second term, $\left(\frac{\Sigma_{deg}(c)}{2m} \right)^2$, represents the expected fraction of edge weight inside community c if edges were placed randomly. Maximizing Q_c (and consequently the overall Q) means finding communities where the internal edge density significantly exceeds random expectation.

2.1.1 Modularity Gain

An important result is the equation to calculate the modularity gain when moving a node to a different community.

$$\Delta Q_{i:a \rightarrow b} = \frac{1}{m} \left[(k_{i,b} - k_{i,a}) - \frac{k_i}{2m} (k_i + \Sigma_{deg}(b) - \Sigma_{deg}(a)) \right] \quad (2.3)$$

The terms in this are defined as follows:

- $\Delta Q_{i:a \rightarrow b}$ is the change in modularity when the node i is moved from the community a to the community b .
- $k_{i,b}$ is the sum of weights of the edges between nodes in community b and node i before moving.
- $k_{i,a}$ is the sum of weights of the edges between nodes in community a and node i before moving (**not including self-loops**).

- k_i is the total sum of the weights of edges (total weighted degree) of node i .
- $\Sigma_{deg}(y)$ is the sum of the weighted degrees of all nodes in the community y (pre movement of the node i into the new community).

2.1.2 Derivation of modularity gain

Equation (2.3) follows from the per-community modularity of Equation (2.2). Moving node i from its community a to a community b changes only the contributions Q_a and Q_b , so

$$\Delta Q_{i:a \rightarrow b} = (Q_a^{\text{after}} + Q_b^{\text{after}}) - (Q_a^{\text{before}} + Q_b^{\text{before}}) = \Delta Q_a + \Delta Q_b.$$

We evaluate the two changes separately. Write $\Sigma_{deg}(a)$ and $\Sigma_{deg}(b)$ for the total weighted degrees of a and b *before* the move (so $\Sigma_{deg}(a)$ includes k_i , since $i \in a$, while $\Sigma_{deg}(b)$ does not), Σ_{in}^a and Σ_{in}^b for their internal edge weights, and $k_{i,a}$, $k_{i,b}$ for the weight of i 's edges into each community.

Community a loses i . Denote the weight of node i 's self-loop as sw_i . Removing i from a drops a 's internal weight by $2k_{i,a} + 2sw_i$ (its $k_{i,a}$ internal edges, each counted twice) and its total degree by k_i , so

$$\begin{aligned} \Delta Q_a &= \left[\frac{\Sigma_{in}^a - 2k_{i,a} - 2sw_i}{2m} - \left(\frac{\Sigma_{deg}(a) - k_i}{2m} \right)^2 \right] - \left[\frac{\Sigma_{in}^a}{2m} - \left(\frac{\Sigma_{deg}(a)}{2m} \right)^2 \right] \\ &= -\frac{k_{i,a}}{m} - \frac{sw_i}{m} + \frac{\Sigma_{deg}(a)^2 - (\Sigma_{deg}(a) - k_i)^2}{(2m)^2} = -\frac{k_{i,a}}{m} - \frac{sw_i}{m} + \frac{2\Sigma_{deg}(a)k_i - k_i^2}{(2m)^2}. \end{aligned}$$

Community b gains i . Symmetrically, adding i to b raises b 's internal weight by $2k_{i,b} + 2sw_i$ and its total degree by k_i , so

$$\begin{aligned} \Delta Q_b &= \left[\frac{\Sigma_{in}^b + 2k_{i,b} + 2sw_i}{2m} - \left(\frac{\Sigma_{deg}(b) + k_i}{2m} \right)^2 \right] - \left[\frac{\Sigma_{in}^b}{2m} - \left(\frac{\Sigma_{deg}(b)}{2m} \right)^2 \right] \\ &= \frac{k_{i,b}}{m} + \frac{sw_i}{m} + \frac{\Sigma_{deg}(b)^2 - (\Sigma_{deg}(b) + k_i)^2}{(2m)^2} = \frac{k_{i,b}}{m} + \frac{sw_i}{m} - \frac{2\Sigma_{deg}(b)k_i + k_i^2}{(2m)^2}. \end{aligned}$$

Combining. Adding the two changes,

$$\begin{aligned}
\Delta Q_{i:a \rightarrow b} &= \frac{k_{i,b} - k_{i,a}}{m} + \frac{(2\Sigma_{deg}(a)k_i - k_i^2) - (2\Sigma_{deg}(b)k_i + k_i^2)}{(2m)^2} \\
&= \frac{k_{i,b} - k_{i,a}}{m} - \frac{2k_i(k_i + \Sigma_{deg}(b) - \Sigma_{deg}(a))}{(2m)^2} \\
&= \frac{1}{m} \left[(k_{i,b} - k_{i,a}) - \frac{k_i}{2m} (k_i + \Sigma_{deg}(b) - \Sigma_{deg}(a)) \right],
\end{aligned}$$

which is exactly Equation (2.3).

2.2 LOUVAIN COMMUNITY DETECTION ALGORITHM

The Louvain algorithm is first presented by Blondel *et al.* (2008). It is one of the most widely used algorithm for community detection as it is simple, fast and scales to very large graphs.

The Louvain algorithm is separated into two phases which repeat in sequence until convergence or a cutoff is reached. One run of the two phases is generally referred to as a *pass*.

2.2.1 Phase 1 : Modularity Optimization

The modularity optimization phase, also known as the local moving phase begins with each node starting in a community with only itself. Sequentially each node in the graph is processed. A node considers moving to each of its neighbour's communities and for each of these moves, the modularity gain is calculated. A node is moved to the community which results in the highest modularity gain (if it is positive).

In one iteration all nodes are processed once. In one modularity optimization phase multiple iterations may occur - this continues until no single move will improve the modularity further.

2.2.2 Phase 2: Community Aggregation

Each community is a collection of nodes at the end of phase 1. In the community aggregation phase, the nodes of each community are combined into one single super-node. If there are multiple edges between communities a and b before this phase, by the end of the phase there is a single edge between the corresponding super-nodes whose weight is the sum of weights of the original edges between those two communities. Edges that are internal to a community (i.e., edges between two nodes of the same community) are collapsed to a self-loop.

By Equation 2.2 we can easily imply that aggregation does not change the modularity of the graph.

```
1   The initial graph is G where each node is in its own community
2   while there is change in the graph:
3       while there is change in the graph:
4           For each node in graph G:
5               Iterate through all neighbouring communities and find
6                   the one with best modularity gain
7                   Move the node into that community if the modularity
9                   gain is positive
10          Aggregate nodes in graph G based on communities formed into
11              the same node. Multiple edges between same communities are
12              added together into one.
```

Listing 2.1: High level pseudocode for Louvain Community Detection Algorithm

2.2.3 A Worked Example

We trace a run of the Louvain community detection on the small graph of Figure 2.1: nine nodes forming three triangles, joined in a ring by the three bridge edges. The graph has $m = 12$ edges, and every node begins in its own community.

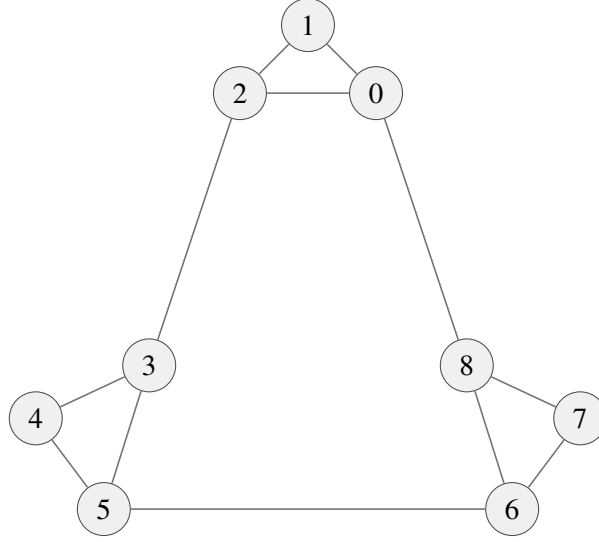


Figure 2.1: A 9-node, 12-edge graph.

Initial modularity

Each node is in its own community. Only the expected-fraction term of equation (2.2) survives. k_i is the degree of the node i . Six nodes carry a bridge and have degree 3, while the three outer-facing nodes have degree 2. The initial modularity is:

$$Q_0 = -\frac{1}{(2m)^2} \sum_i k_i^2 = -\frac{6 \cdot 3^2 + 3 \cdot 2^2}{24^2} = -\frac{66}{576} \approx -0.115.$$

Pass 1 - (local moving phase).

Local moving visits each of the nodes. A node leaving its own singleton community $\{i\}$ has $k_{i,a} = 0$ and $\Sigma_{deg}(a) = k_i$, so the gain of Equation (2.3) for moving it into a community b simplifies to

$$\Delta Q_{i \rightarrow b} = \frac{1}{m} \left[k_{i,b} - \frac{k_i \Sigma_{deg}(b)}{2m} \right],$$

where $k_{i,b}$ is the weight of i 's edges into b and $\Sigma_{deg}(b)$ the total degree of b .

The local-moving phase considers the node 1 first ($k_1 = 2$). Its neighbours are 0 and 2,

each still a singleton of degree 3. Moving into $\{0\}$ (one edge, so $k_{1,\{0\}} = 1$) gives

$$\Delta Q_{1 \rightarrow \{0\}} = \frac{1}{12} \left[1 - \frac{2 \cdot 3}{24} \right] = \frac{1}{12} \cdot \frac{3}{4} = 0.0625,$$

and moving into $\{2\}$ gives the same 0.0625 by symmetry. The tie is broken arbitrarily - say node 1 joins $\{0\}$, forming the community $\{0, 1\}$ (total degree $\Sigma_{deg} = 5$).

Node 2 ($k_2 = 3$) is visited next. Its neighbours now lie in two communities: $\{0, 1\}$, reached by the two edges $(2, 0)$ and $(2, 1)$ so $k_{2,\{0,1\}} = 2$, and the singleton $\{3\}$, reached by the bridge $(2, 3)$ so $k_{2,\{3\}} = 1$. The two candidate modularity gains are

$$\Delta Q_{2 \rightarrow \{0,1\}} = \frac{1}{12} \left[2 - \frac{3 \cdot 5}{24} \right] \approx 0.115, \quad \Delta Q_{2 \rightarrow \{3\}} = \frac{1}{12} \left[1 - \frac{3 \cdot 3}{24} \right] \approx 0.052.$$

Node 2 moves into the $\{0, 1\}$ community which then becomes $\{0, 1, 2\}$.

The remaining nodes are processed one after another: 3, 4, 5 coalesce into one community and 6, 7, 8 into another. They are pulled together by strong within-triangle edges while the bridges are too weak. The pass ends with the three communities of Figure 2.2. Each community having 3 internal edges and total degree 8. The modularity has climbed from -0.115 to

$$Q = 3 \left[\frac{3}{12} - \left(\frac{8}{24} \right)^2 \right] \approx 0.42,$$

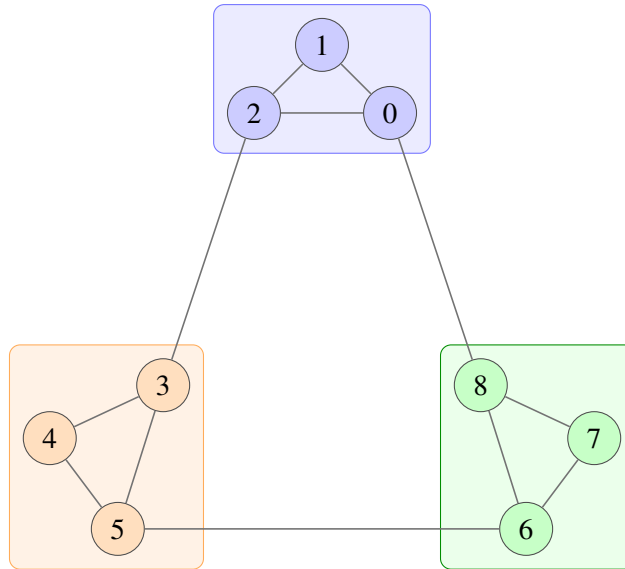


Figure 2.2: After the local-moving phase.

Pass 1 - (aggregation phase)

Each community is now aggregated into a single super-node. The three internal edges of a triangle collapse into a self-loop (each of weight 3), and the bridge edges become unit-weight edges between super-nodes. The result (Figure 2.3).

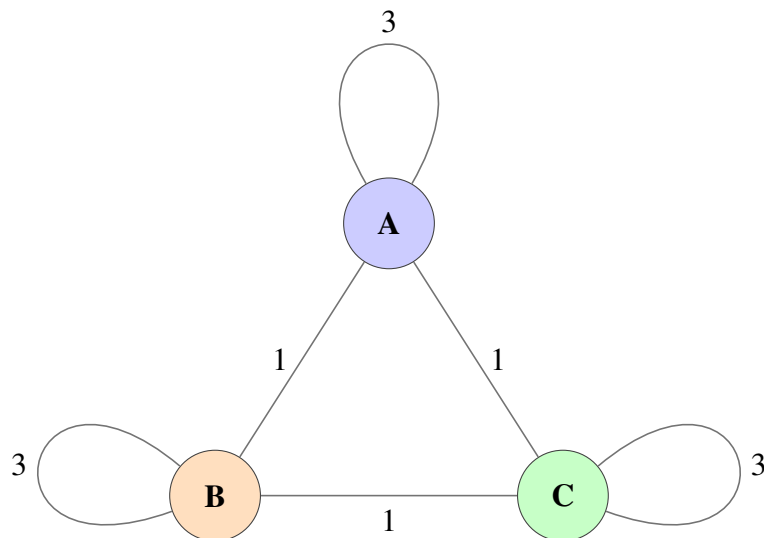


Figure 2.3: After aggregation.

Pass 2 - local moving phase.

A second pass now runs local moving on the three node graph. No move improves Q , the partition is unchanged across the pass.

To see this concretely, consider moving node A into the community of node B.

Apply the Equation (2.3). Here $k_{A,\{A\}} = 0$ as self-loops are not considered, and $k_{A,\{B\}} = 1$; the two communities have equal total degree, $k_A = \Sigma_{deg}(\{A\}) = \Sigma_{deg}(\{B\}) = 8$ (self-loops are counted twice). Substituting,

$$\begin{aligned}\Delta Q_{A:\{A\} \rightarrow \{B\}} &= \frac{1}{m} \left[(k_{A,\{B\}} - k_{A,\{A\}}) - \frac{k_A}{2m} (k_A + \Sigma_{deg}(\{B\}) - \Sigma_{deg}(\{A\})) \right] \\ &= \frac{1}{12} \left[(1 - 0) - \frac{8}{24} (8 + 8 - 8) \right] = \frac{1}{12} \left[1 - \frac{8}{3} \right] = -\frac{5}{36} \approx -0.139\end{aligned}$$

The gain is *negative*, so the move is rejected. All other possible moves are also symmetric and hence they are also rejected. The community partition reached is stable - the algorithm terminates with these three communities.

2.3 NVIDIA GPUS

NVIDIA GPUs originally meant for graphics processing have evolved into a dominant computing centre for AI, scientific computing and high performance workloads.

2.3.1 Philosophy

CPUs have a relatively small number of powerful cores each with increasingly sophisticated builds - deep pipelines, branch prediction, out-of-order execution. All this results in low latency computations - a single computation will finish as quickly as possible.

This highly contrasts with GPUs which are throughput optimized. Their goal is not to finish a single thread as quickly as possible but complete an enormous amount of work

by having an intake of a huge amount of threads.

For a concrete example:

1. NVIDIA H100 GPU: Contains 132 streaming multiprocessors each with 128 CUDA cores and 4 tensor cores which results in 16896 CUDA cores and 528 tensor cores. Each SM has 4 warp schedulers which can issue instructions to a 32-thread warp every clock cycle. The clock speeds are around 1.8 GHz. There is no branch prediction or speculative execution per thread.
2. High-end Xeon 6980P(Granite Rapids): 128 performance cores with each capable of running 2 threads via hyperthreading. Base clock speed is 2Ghz and can boost to 3.9Ghz on a single thread.

2.3.2 Architecture

Their architecture revolves around two interlocking layers: the physical hardware organization and the computation model. The hardware includes the transistors, memory hierarchy and specialized cores. The computation model is what is provided as an abstraction to the programmer.

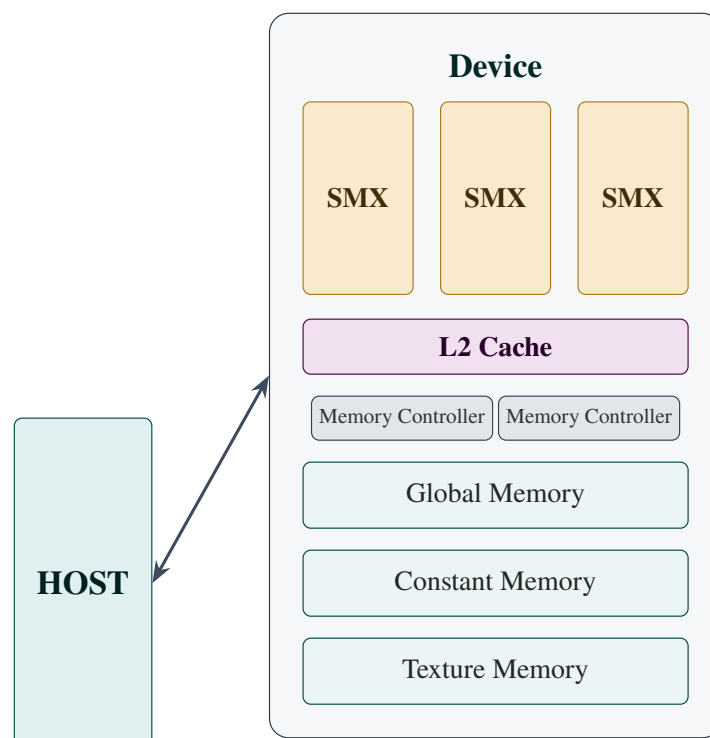


Figure 2.4: GPU Device Architecture

Streaming Multiprocessor

This is roughly analogous to a CPU core but each SM can run a large number of threads (100s). Each SM contains its own registers, caches, schedulers and execution units.

There is specialized hardware in each SM: CUDA cores(general purpose), tensor cores (matrix multiplication), ray tracing cores, special function units, registers, L1 cache and shared memory.

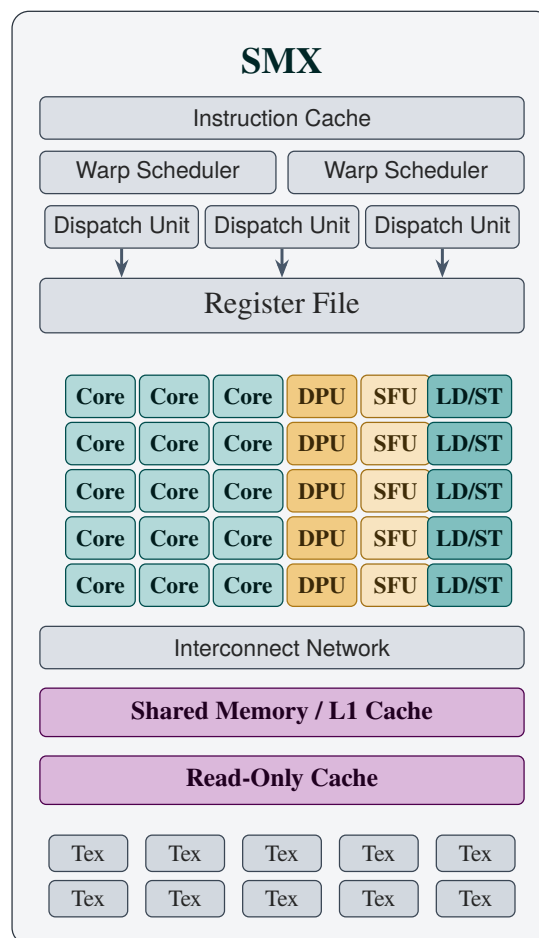


Figure 2.5: Streaming Multiprocessor Architecture

Memory Hierarchy

Each SM has 64K 32-bit *registers* on Hopper-class GPUs. *Shared memory and L1 cache* are a unified pool in each SM. Shared memory is used for communication within a thread block and is managed by the programmer whereas L1 cache is managed by the hardware.

There are some provisions to change the ratio of these two types of memory. Hopper has 256KB of this combined pool per SM. *L2 cache* is shared across SMs. On Hopper-class GPUs there is 50MB of L2 cache.

The H100 also ships with upto 80GB of *high bandwidth memory* which is a type of DRAM.

Beyond this, GPUs can access CPU memory via unified memory or pinned memory techniques via the PCI-express bus. NVLink is a hardware bridge for direct, high-speed communication between NVIDIA GPUs.

2.4 CUDA PROGRAMMING MODEL

CUDA (Compute Unified Device Architecture) is NVIDIA's parallel programming model. It is the abstraction for programmers to write code for GPUs. Efficiently porting algorithms to NVIDIA GPU requires proficiency in CUDA's execution model, memory hierarchy and abstraction primitives to exploit the hardware, making *CUDA expertise necessary for this project*.

CUDA allows developers to write specialized functions known as "kernels" in C/C++ code. A kernel is declared with the function execution space identifier `__global__`. The `KERNEL_NAME<<< GRID_DIM, BLOCK_DIM >>>` syntax is used to call the kernel from the host (CPU).

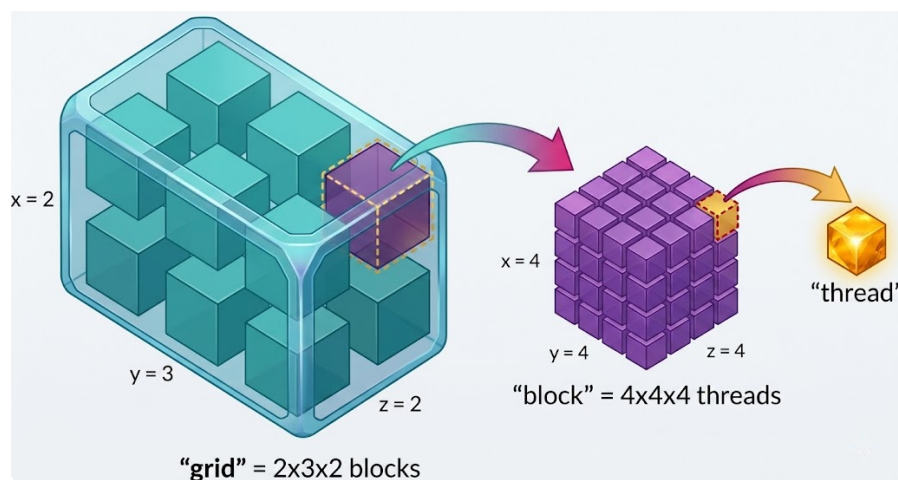
```
1 __global__ void dkernel() {  
2     // GPU Processing  
3 }  
4  
5 // Kernel launched with  
6 // dkernel<<<GRID_DIMENSIONS, BLOCK_DIMENSIONS>>>();
```

Listing 2.2: kernel function syntax

These kernels are executed by multiple threads in parallel. Threads are organized into

warps, blocks and grids. Each warp contains 32 threads. Threads in a warp execute in SIMT (single instruction, multiple thread) fashion. When there is control divergence between threads of a warp it results in performance penalties.

Each block generally contains upto 1024 threads which can be arranged into a 3-dimensional array of threads and each grid can be thought of as a 3-dimensional array of blocks. When a kernel is started it launches one entire grid. The size of the grid can be specified in parameters by the programmers at launch.



In this section we discuss some aspects of the CUDA programming model that would assist in following our static and dynamic Louvain implementations. We discuss:

- Launching a kernel
- Memory management
- Thread indexing
- Functions and their execution-space specifiers
- The thrust library
- Synchronisation Primitives

2.4.1 Launching A Kernel

The following kernel is launched with a 1x1x1 grid with each block having 32 threads in a 32x1x1 thread-block configuration. `dim3` is a 3 size vector which is used to specify x,

y and z dimensions for the grid and block arrays. `cudaDeviceSynchronize()` blocks the CPU until the GPU finishes - this is required as kernel launches are asynchronous and the program on the CPU could exit before the kernel finishes running.

The output of running this program is "Hello World" 32 times.

```
1  __global__ void dkernel() {
2      printf("Hello World\n");
3  }
4
5  int main() {
6      dim3 grid(1, 1, 1);
7      dim3 block(32, 1, 1);
8      dkernel<<<grid, block>>>();
9      cudaDeviceSynchronize();
10     return 0;
11 }
```

Listing 2.3: The CUDA Hello World

2.4.2 Memory

Memory allocation on the GPU can be done with a memory allocation function with the following signature:

```
cudaError_t cudaMalloc(void** devPtr, size_t size)
```

This allocates `size` bytes and sets value pointed to by `devptr` to the GPU memory address where the allocation begins.

Memory between the CPU and GPU can be moved using a memory copy function with the following signature:

```
cudaError_t cudaMemcpy(void* dst, const void* src, size_t count, cudaMemcpyKind
                        kind)
```

This copies `count` bytes from `src` to `dst`, with the direction specified by `kind`. `kind` is

one of `cudaMemcpyHostToDevice` (CPU to GPU), `cudaMemcpyDeviceToHost` (GPU to CPU), `cudaMemcpyDeviceToDevice` (GPU to GPU), `cudaMemcpyDefault`.

2.4.3 Another kernel

Several built-in indexing variables (each with x, y and z components) are accessible inside a kernel function:

1. `gridDim` - The size of the grid.
2. `blockIdx` - Provides the x, y and z indices of the thread-block that accesses it within the grid.
3. `blockDim` - The size of a block.
4. `threadIdx` - Provides the x, y and z indices of the thread that accesses it within the block.

The below code snippet illustrates the memory calls and thread organisation.

```
1 // Each thread processes one pixel
2 // The volume is stored as a flat 1D array indexed in (x, y, z) order
3
4 __global__ void scaleVolume(float* volume, int x_threshold, int
5     y_threshold, int z_threshold, float factor)
6 {
7     // Global 3D coordinates of this thread
8     int x = blockIdx.x * blockDim.x + threadIdx.x;
9     int y = blockIdx.y * blockDim.y + threadIdx.y;
10    int z = blockIdx.z * blockDim.z + threadIdx.z;
11
12    // If the thread is outside the box do nothing
13    if (x < x_threshold && y < y_threshold && z < z_threshold) {
14        // Flatten coordinates
15        int idx = x + y * Nx + z * Nx * Ny;
16        volume[idx] *= factor;
17    }
18 }
19
20 int main()
21 {
22     // Nx, Ny and Nz are #define-d
23     int size = Nx * Ny * Nz * sizeof(float);
```



```

22     float* volume = malloc(size);
23
24     float* d_volume;
25     cudaMalloc(&d_volume, size);
26     cudaMemcpy(d_volume, volume, size, cudaMemcpyHostToDevice)
27
28     // Define a 3D block: 8 x 8 x 8 = 512 threads per block
29     dim3 threadsPerBlock(8, 8, 8);
30
31     // Define a 3D grid: enough blocks to cover the entire volume
32     dim3 numBlocks((Nx + threadsPerBlock.x - 1) / threadsPerBlock.x,
33                   (Ny + threadsPerBlock.y - 1) / threadsPerBlock.y,
34                   (Nz + threadsPerBlock.z - 1) / threadsPerBlock.z);
35
36     // Launch: A grid of blocks each with 512 threads
37     scaleVolume<<<numBlocks, threadsPerBlock>>>(d_volume, Nx, Ny, Nz,
38         2.0f);
39
40     cudaDeviceSynchronize();
41     cudaFree(d_volume);
42     return 0;
43 }

```

Listing 2.4: 3D volume processing illustrating thread and block organization and memory calls

2.4.4 Functions

In CUDA, each function is annotated with an *execution space specifier* which conveys where the function runs (CPU : host or the GPU : device) and from where it can be called. The three specifiers are: `__global__`, `__device__`, and `__host__`. The default is `__host__` when none is given. `__global__` has been mentioned previously when discussing kernels.

A function marked `__host__` runs on the host and is callable only from host code.

A function marked `__device__` runs on the device and may be called *only* from other device code (i.e. from a `__global__` kernel or another `__device__` function). It is like a helper function for the GPU code. They are typically inlined by the compiler.

```

1  __device__ float getIdX(int x, int y, int z) {

```

```

2     return x + y * Nx + z * Nx * Ny;
3 }
4
5 __global__ void scaleVolume(float* volume, int x_threshold, int
6     y_threshold, int z_threshold, float factor)
7 {
8     // Global 3D coordinates of this thread
9     int x = blockIdx.x * blockDim.x + threadIdx.x;
10    int y = blockIdx.y * blockDim.y + threadIdx.y;
11    int z = blockIdx.z * blockDim.z + threadIdx.z;
12
13    // If the thread is outside the box do nothing
14    if (x < x_threshold && y < y_threshold && z < z_threshold)
15    {
16        volume[getIdx(x, y, z)] *= factor;
17    }
18 }

```

Listing 2.5: A `__device__` helper called from a kernel

The `__host__` and `__device__` specifiers can be used together to annotate a single function. This instructs the compiler to generate the function machine code on both the host and device so that it can be called from both contexts without writing the source code twice.

```

1 __host__ __device__ float double(float v) {
2     return v*2
3 }

```

Listing 2.6: A function compiled for both host and device

Specifier	Executes on	Callable from	Returns value?
<code>__global__</code>	Device	Host (via <<<, >>>)	No (void only)
<code>__device__</code>	Device	Device	Yes
<code>__host__</code> (default)	Host	Host	Yes
<code>__host__ __device__</code>	Both	Both	Yes

Table 2.1: CUDA function execution space specifiers

2.4.5 Thrust: A library

Thrust is a C++ template library which provides many parallel algorithms and data structures that comes with the CUDA toolkit. It is similar in spirit to the C++ Standard

Template Library (STL) but for GPUs. It is portable - almost identical source code runs on either the host or the device based on the container type.

Similar to `std::vector` thrust has two vectors that reside in different memory spaces:

- `thrust::host_vector<T>` - resides in host memory.
- `thrust::device_vector<T>` - resides in device memory.

Thrust algorithms operate on iterator ranges similar to STL. The type of the iterator determines where the algorithm runs. This is why generally iterators passed to a single algorithm must belong to the same device.

Functors

Functors are C++ objects that overload the `operator()` which allows objects to be called like a function while holding state in its data members. Thrust provides many predefined functors such as `thrust::negate`, `thrust::modulus` and `thrust::plus`.

Custom functors can also be defined by programmers. The call operator should be annotated with `__host__` `__device__` so it can run on the host or the device.

```
1 struct scale {  
2     const float scale_const;  
3     scale(float a) : scale_const(a) { }  
4  
5     __host__ __device__  
6     float operator()(const float& x) const {  
7         return scaler * x;  
8     }  
9 };
```

Listing 2.7: scaling functor

Some algorithms

`thrust::transform` applies a unary or binary operation element-wise over an input range. A unary operations maps one input to one output whereas a binary operation combines two inputs into one output.

A reduction applies a binary operation successively over a range, collapsing it into a single value. On the GPU this is completed in $O(\log n)$ parallel steps. By default the sum functor is used for reductions.

```
1 // Sum values of array
2 int x = thrust::reduce(vec.begin(), vec.end());
3
4 // Explicit initial value and functor
5 y = thrust::reduce(dvec.begin(), dvec.end(), 0, thrust::plus<int>());
```

Listing 2.8: Using thrust::reduce

A scan (also called prefix sum) produces a vector output where each element is the sum of all previous elements. There are two variants of this in thrust:

```
1 int data[6] = {1, 0, 2, 2, 1, 3};
2
3 thrust::inclusive_scan(data, data + 6, data);
4 // data is now {1, 1, 3, 5, 6, 9}
5
6 // (on the original input)
7 thrust::exclusive_scan(data, data + 6, data);
8 // data is now {0, 1, 1, 3, 5, 6}
```

Listing 2.9: thrust scan example

- `inclusive_scan` - The i^{th} element in the output includes input element i .
- `exclusive_scan` - The i^{th} element in the output excludes input element i .

Thrust provides sorting algorithms as well. `thrust::sort` sorts a range in place. `thrust::sort_by_key` sorts a key range and places the corresponding values in the same order.

```
1 int keys[4] = {2, 5, 3, 7};
2 char values[4] = {'a', 'b', 'c', 'd'};
3 thrust::sort_by_key(keys, keys + 4, values);
4 // keys is now {2, 3, 5, 7}
5 // values is now {'a', 'c', 'b', 'd'}
```

Listing 2.10: sort-by-key example

We use many of these thrust algorithms in our implementations of static and dynamic

Louvain in CUDA, for instance the thrust sort and reduce functions are utilised in the **aggregation of communities** into super-nodes.

2.4.6 Synchronisation Primitives

A kernel may launch thousands of threads at once, yet many algorithms need them to coordinate with each other: to combine partial results into one value, to agree on which thread updates a shared location, or to make one thread's writes visible to another. CUDA provides several families of primitives for this - *atomic operations*, *barriers*, and *memory fences*. Our static and dynamic Louvain kernels make use of these primitives to ensure correctness.

Atomic operations

An atomic operation performs a *read-modify-write* on a memory location as a single indivisible step. If many threads atomically update the same address the hardware serialises them, so no update is lost. CUDA provides `atomicAdd`, `atomicSub`, `atomicExch` (swap), `atomicMin/atomicMax`, the bitwise `atomicAnd/atomicOr/atomicXor`, and the compare-and-swap `atomicCAS(addr, expected, val)`, which writes `val` only if the current value equals `expected` and, in every case, returns the value that was there before.

A simple `counter += 1` from many threads is a race. Each thread reads the old value, adds, and writes back, so simultaneous updates clobber one another. `atomicAdd(&counter, 1)` makes the increment safe.

```
1 __global__ void total(const float* a, int n, float* out) {
2     int i = blockIdx.x * blockDim.x + threadIdx.x;
3     if (i < n) atomicAdd(out, a[i]); // indivisible read-add-write
4 }
```

Listing 2.11: Accumulating into one counter with `atomicAdd`

In our implementation `atomicAdd` is used to accumulate the **total modularity gain** of a local-moving iteration into a single device counter.

Locks from compare-and-swap

`atomicCAS` can be used to build a *lock*. A lock is an `int` flag that is 0 when free: a thread acquires it by atomically swapping 0 for 1, spinning until it succeeds, and releases it by writing 0 back with `atomicExch`.

```
1 while (atomicCAS(&lock, 0, 1) != 0) { }    // spin until free
2 // ... critical section ...
3 atomicExch(&lock, 0);                      // release the lock
```

Listing 2.12: A spin-lock built atomics

Our local-move kernel uses this construct to lock the **pair of communities a vertex moves between**, so that the move’s modularity gain can be re-checked against frozen community totals before it is committed.

Memory fences

The `__threadfence()` intrinsic makes all of the calling thread’s previous global-memory writes visible to other threads before any of its subsequent writes. Our kernel calls `__threadfence()` **after a committed move**. This will be explained further in the report.

Barriers: block and grid

`__syncthreads()` is a barrier across the threads of a single *block*: every thread waits there until all threads in the block have arrived.

Synchronising across an *entire grid* (all threads of all blocks) is more difficult, because the blocks of a launch are not guaranteed to be resident (remember you can launch many more blocks than can fit on the GPU’s cores at once) at the same time. CUDA’s cooperative-groups API does offer a device-wide barrier, `grid.sync()`, but it requires a special *cooperative kernel launch* and restricts the grid to what fits on the device at once.

While this was used in the UGRC implementation **our implementation deliberately avoids it**. Rather than one persistent kernel that barriers the whole grid after every

local-moving iteration, the host *relaunches* the kernel once per iteration.

2.4.7 Error Checking

Kernel launches and many runtime calls can fail - resource limits could be reached, or a non existed memory location could be accesses.

CUDA provides support for error checking. Runtime API functions return a `cudaError_t`; for asynchronous kernel launches, the error is retrieved afterwards with `cudaGetLastError()` (which clears the error) or `cudaPeekAtLastError()` (which does not). Below is a common macro for error checking:

```
1  #define CUDA_CHECK(call) \
2      do { \
3          cudaError_t err = (call); \
4          if (err != cudaSuccess) { \
5              fprintf(stderr, "CUDA error %s at %s:%d\n", \
6                  cudaGetErrorString(err), __FILE__, __LINE__); \
7              exit(EXIT_FAILURE); \
8          } \
9      } while (0)
10
11  CUDA_CHECK(cudaMalloc(&d_data, size));
12
13  dkernel<<<grid, block>>>(d_data);
14  CUDA_CHECK(cudaGetLastError()); // This catches launch
    configuration errors
15  CUDA_CHECK(cudaDeviceSynchronize()); // This catches errors during
    execution of the kernel
```

Listing 2.13: CUDA_CHECK macro for errors

Because launches are asynchronous, `cudaGetLastError()` immediately after a launch catches *launch-time* faults (e.g. an invalid grid), while a following `cudaDeviceSynchronize()` surfaces *run-time* faults that occur during kernel execution.

2.4.8 The NVCC compiler

NVIDIA Corporation (2026). CUDA source files (usually have an `.cu` extension) mix

host and device code. A standard C++ compiler cannot process .cu files. NVCC (NVIDIA CUDA Compiler) is the CUDA toolkit's compiler driver that handles this mixed source.

Virtual and Real Architectures

NVCC distinguishes two architecture targets, set at compile time:

- **Virtual** (compute_XX) - the feature/capability level that the PTX assembly is targeting.
- **Real** (sm_XX) - the GPU generation for which SASS (machine code) is produced.

The sm_XX is the *compute capability* of the target GPU (e.g. sm_70 for Volta, sm_75 for Turing, sm_80/sm_86 for Ampere, sm_90 for Hopper). PTX is forward-compatible and a binary can carry only PTX and let the driver JIT-compile it for the GPU present in the hardware, or carry precompiled SASS for the specific GPU architecture. It is even possible for a single fat binary to combine both.

```
bash
```

```
# Simple
$ nvcc program.cu -o program

# Target one architecture
$ nvcc -arch=sm_75 program.cu -o program

# Fat binary
$ nvcc -gencode arch=compute_70,code=sm_70 -gencode
  arch=compute_80,code=sm_80 program.cu -o program
```


CHAPTER 3

LITERATURE REVIEW

A survey (literature review) of existing research on the Louvain algorithm is done and reported below. There exist a wide array of implementations ranging from single core CPUs, multi-core CPUs, GPUs, CPU-GPU hybrid systems and distributed systems. Studying sequential implementations will give us more intuition about the foundational algorithm and surveying parallel implementations will assist in building ideas which can be carried to our implementation. We talk about various aspects such as:

- Interesting aspects of modularity.
- The original paper proposing the Louvain method.
- The mapping of the output of the Louvain method to meaning.
- Various papers implementing Louvain targeting different machines.
- Improvements and Modifications to the algorithm which may or may not be specific to the computation device. Algorithmic heuristics applied would be interesting as well.
- Parallel computing and GPU specific optimizations.

3.1 MODULARITY

Brandes et al. (2008) formalise modularity, proving that maximizing it is NP-hard. This justifies the investigation of approximation and heuristic based algorithms like Louvain for community detection when optimizing for modularity.

Some interesting lemmas and results regarding modularity from this paper are stated below:

- Isolated nodes do not impact modularity.

- A clustering with maximum modularity does not have a cluster which consists of only one node with degree 1.
- There must be a clustering with maximum modularity where each cluster consists of a connected subgraph.
- The maximum modularity clustering of a clique consists of a single cluster and for cycles of size n consists of approximately $\sqrt{n}$ clusters each of size $\sqrt{n}$.
- A clustering of a bipartite graph $K_{a,b}$ into two clusters $C = \{C_a, C_b\}$ has the theoretically least possible modularity of $-1/2$.

Some interesting counter-intuitive behaviors from the paper are stated below:

- Modularity can exhibit non local behavior as presented by adding a vertice to graphA 3.1a to produce graphB 3.1b. The rightmost black node shifts to the white cluster.
- Sensitivity: A clique's (with n nodes) maximum modularity (of zero) is achieved when there is only one cluster. When we add a leaf for each vertex, then the optimal clustering is n clusters each with 2 nodes. This is depicted in graphC.
- The clusters drastically change in behavior by just doubling the graph from graphC 3.1c to graphD 3.1d.

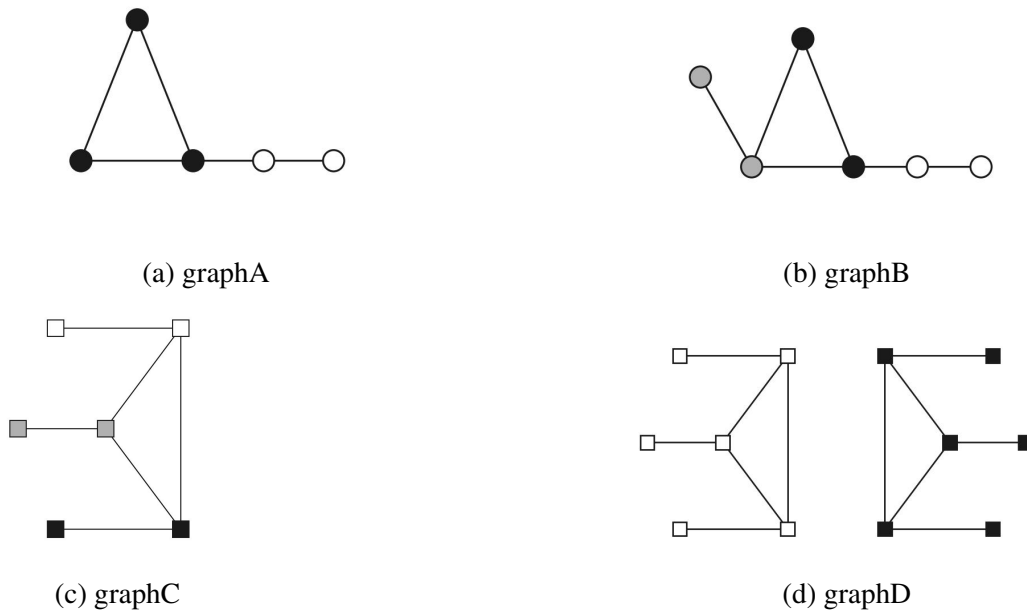


Figure 3.1: Adapted from Brandes *et al.* (2008), © 2008 IEEE.

3.2 THE ORIGINAL PAPER

Blondel *et al.* (2008) is the original paper proposing the Louvain community detection algorithm. It is a heuristic algorithm which they claim outperforms all existing community detection methods in terms of computation time and additionally generating very good communities measured by modularity.

An existing greedy algorithm **Clauset *et al.* (2005)** is mentioned, however this produced communities with modularity worse than methods like simulated annealing and tends to result in the formation of large super-communities. This tends to slow down the algorithm which results in impracticality of using this on graphs of more than 1 million nodes.

Unlike many previous methods, Louvain experimentally runs in nearly linear time on typical and sparse data which allows the analysis of very large graphs. Benchmarking has been on graphs upto 1 billion edges. An interesting fact about the benchmarking on large graphs is that there are atmost 5 aggregation passes.

They run the algorithm on a Belgian mobile network and later on ad-hoc modular networks. There are interesting discussions regarding the community output on the belgian phone network. Six levels of hierarchy are found, with language being a major separating factor in final communities formed. The paper discusses that the intermediate communities in the hierarchy should also be meaningful. Various qualitative supportive arguments are provided:

- Intermediate partitions correspond to local maxima of modularity. There can be no increase by moving one node to a different community.
- Final community structure has a high modularity in a broad range of systems.
- A self-consistency thought experiment. Take a community C from the final pass, isolate it (delete all edges leaving C), treat the result as its own network, and re-run Louvain on it. The expectation is you would recover the same sub-communities that the penultimate pass produced for C . They don't run this experiment in the paper but argue it should work, due to the local-optimization nature of each step.

Quoting a paragraph in the paper regarding heuristics "It is interesting to note that the speed of our algorithm can still be substantially improved by using some simple heuristics, for instance by stopping the first phase of our algorithm when the gain of modularity is below a given threshold or by removing the nodes of degree 1 (leaves) from the original network and adding them back after the community computation. The impact of these heuristics on the final partition of the network should be studied further, as well as the role played by the ordering of the nodes during the first phase of the algorithm."

Some other caveats and notable points are below:

- *Order dependence*: The result depends on which vertex you visit first in the local-moving phase. The authors say empirical experiments show this rarely affects the modularity score significantly, but it can affect which specific partition you get when several near-optimal partitions exist. They do not propose a fix; they just note it.
- *The resolution limit* Fortunato and Barthélemy (2007). The authors acknowledge that modularity itself, as an objective, can't see communities smaller than $\sqrt{2m}$. The hierarchy point above is the workaround.
- *No quality guarantee*: Like all greedy methods, Louvain finds a local maximum of modularity, not a global one. The authors don't claim optimality - just that empirically the local maxima it finds are very good.

The details of the Louvain algorithm and modularity are discussed in 2.1 and 2.2.

3.3 CPU-BASED STATIC LOUVAIN ALGORITHMS

3.3.1 A heuristics deep dive

Rotta and Noack (2011) organizes existing and new single-level and multilevel heuristics into a coherent design space and run a systematic experimental comparison across 62 real-world graphs. The payoff: a previously unevaluated combination - single-step cluster joining with their new Z-Score prioritizer, plus multilevel refinement by local vertex moving - turns out to be competitive with the best published modularity-clustering methods at the time.

The following are the dimensions considered:

- Coarsening Algorithm
- Criterion for choosing the joined clusters or moved vertices in coarsening phase
- The refinement algorithm
- Criterion for choosing the clusters or moved vertices in the refinement phase
- Number of coarsening levels

3.3.2 Multicore CPU Implementations

These are mainly OpenMP shared memory implementations.

Grappolo

Lu *et al.* (2015) is a canonical shared-memory parallel Louvain baseline implemented in C++ and OpenMP. The paper discusses obstacles to parallelizing the Louvain algorithm.

If vertex movements are done in parallel there is no guarantee that the net modularity gain is positive. An example is discussed: nodes i and j are both neighbours of a third node k . The threads processing them, decide in parallel to move to the community containing node k . The moves occur but the net modularity gain from the combination of these moves can be negative as the final community structure is not the same as what either thread considered while calculating the modularity gain - each only considered a *single node* move. This also implies that we cannot guarantee algorithm termination - as modularity is no longer guaranteed to be non-decreasing.

They identify another key obstacle to parallelism - concurrent moves producing "community swaps" (two vertices each try to follow the other into each other's community and oscillate) - and resolve it using a minimum-label-breaking heuristic.

Vertex colouring can be used as a parallelism enabler. They run a one-time greedy vertex colouring before local-moving. Vertices of the same colour are guaranteed non-adjacent,

so their move decisions cannot affect each other's modularity gain - they are independent in the strict sense. The local-moving phase then iterates over colours sequentially but processes all vertices of one colour in parallel. This trades a one-time cost of colouring for race-free move evaluation. There exists a tradeoff that graphs with high chromatic number (dense graphs) get less parallelism per colour.

Vertex following preprocessing heuristic: Single degree vertices are merged a priori into their respective neighbouring vertex.

They also introduce several optimization toggles iteration-threshold, that later GPU work has largely adopted.

NetworKit

Staudt and Meyerhenke (2016) engineer a parallel Louvain variant for the NetworKit toolkit, targeting massive networks with careful attention to load balancing, vertex iteration order, and cache behaviour. They split the local-moving phase into batches and use OpenMP task parallelism.

Three standalone parallel algorithms are presented in their paper:

- PLP (Parallel Label Propagation): The base algorithm is introduced by **Raghavan et al. (2007)**. Initially each node is assigned distinct labels. In each iteration of the algorithm, each node modifies its label to the most frequent label in its neighbourhood which eventually leads to a global stable consensus. In the parallel implementation they accept race conditions - and even mention that this actually may assist in the ensemble setting due to random variations.
- PLM (Parallel Louvain Method)
- PLMR (Parallel Louvain Method - Refinement): Follows up on the work of **Rotta and Noack (2011)** (previously discussed in this report). There is a second move phase in PLMR after recursively coarsening.

In addition to these they present a two-phase approach inspired by ensemble learning, in which the solutions of the base algorithms are combined to form core communities. The graph is coarsened according to these core communities, following which it is assigned

to a single final algorithm. This is named EPP (Ensemble Pre Processing) method. PLP is applied as the base algorithms and PLMR as the final algorithm.

A benchmark on uk-2007-05 graph with 3.3 billion edges with the 16-core (32 hyper threads) PLM is reported to complete in under 3 minutes. Experimental graphs used are from the 10th DIMACS Implementation Challenge and SNAP.

APLM - Adaptive Parallel Louvain Method

Fazlali et al. (2017) discusses two alternative methods to decompose the Louvain algorithm for parallelization.

1. Compute the modularity gain of adding a neighbour node to a community in parallel.
2. Run a separate thread for each neighbour node to calculate the gained modularity.

A key novelty they introduce is adaptability. The core idea is that the granularity of the tasks which are assigned to threads is based on the number of idle cores.

Proposed Method: Calculate the number of community neighbour nodes which may be added to the community and obtain the idle cores using and API. These pieces of information together are used to decide the granularity of tasks.

Baseline algorithms used for comparison include PLM from NetworKit and CADS from *Moradi et al. (2015)*. APLM beats both algorithms on all 5 graphs tested on.

Something I hypothesize is that similar adaptive methods can be applied when parallelizing on GPUs - such as adapting based on the number of streaming multiprocessors on the specific GPU model being used.

GVE-Louvain

Sahu (2023) presents one of the most efficient multicore implementations of the Louvain algorithm. It is reported to outperform Vite, Grappolo, NetworKit Louvain, and cuGraph Louvain (A100 GPU) by 50×, 22×, 20×, and 5.8× and performs on average 1.6× for

every doubling of threads.

A foundational design choice that GVE-Louvain makes is that it runs an asynchronous version of Louvain - threads work independently on different parts of the graph without synchronization at the local moving steps. Per community $\sum_{tot}$ values are updated with atomics and there is no old-label & new-label arrays.

Their approach clearly mentions the optimizations considered:

1. OpenMP loop scheduling can be adjusted between static, dynamic, guided or auto. Dynamic reported to work the best due to load balancing when degree distribution of vertices is not uniform.
2. Restrict iterations for the local moving phase.
3. Threshold for deciding convergence of a local moving phase. Adjust initial pass threshold.
4. This threshold is also not kept constant but is scaled for consecutive passes. The first pass is generally the most expensive and this reduces runtime for it. (10× scaling is used)
5. Aggregation tolerance where we stop the Louvain pass after a certain percentage of super vertex merges have been finished. This avoids further merges which supposedly have minimal impact.
6. Vertex pruning. Mark neighbours of vertices when moved to new communities as "to be processed". Mark the moved vertex as "processed".
7. Parallel prefix sum for aggregation phase and usage of CSR representation of the graph.
8. Usage of Far-KV hashtable instead of Close-KV or `std::map`. Close-KV is a hashtable which is allocated contiguously. Although different threads process different parts of the graph, it is possible that two threads may access the same cache line - this leads to false sharing in caches.

Takeaways include:

- The asynchronous versus synchronous tradeoff which would be crucial for the algorithm ported to GPUs as well. Due to the sheer amount of parallelism on GPUs we would likely wish to go with a asynchronous design.

- The structured optimizations provide an easy reference to be applied to our implementation.

3.3.3 Vite

[Ghosh *et al.* \(2018b\)](#) is a MPI+OpenMP distributed-memory implementation of the Louvain algorithm. It is one of the only open-source available MPI implementations.

[Ghosh *et al.* \(2018a\)](#) follows the previous work with a primarily evaluation based paper.

Two performance heuristics are proposed:

- **Threshold Cycling:** A threshold is used in the Louvain algorithm to decide the termination. That is, if the net modularity gain between two passes is below a certain value the algorithm is said to have achieved convergence and subsequently terminates.

The example used in the paper is that for passes 0–2, 3–6, 7–9 and 10–12 respective thresholds of 10^{-3} , 10^{-4} , 10^{-5} and 10^{-6} . This pattern is again repeated from phase 13 and so on.

- **Early Termination:** An *active* probability value is assigned to each vertex which is reduced by a factor of $1 - \alpha$ for each iteration in the local-moving phase when the vertex does not move to different community. It is set to 1 if the vertex moves to different community. This percentage is used to decide whether the vertex will participate in the computation in the local-moving phase to possibly change community affiliation.
- **Incomplete Colouring:** The Jones-Plassman algorithm is used to colour the vertices. All vertices of a single colour can be processed concurrently as it is guaranteed that there is no edge between any two vertices of the same colour.

On single-core benchmarking against Grappolo, the performance is reported to be around 2-6x worse on the soc-friendster graph of 1.8B edges. However, when scaled out to 4K processes on 256 nodes, their implementation achieves a 7x speedup over Grappolo.

Takeaways include:

- The threshold cycling, early termination and incomplete colouring heuristics.
- In the evaluation section Strong and weak scaling analysis is done.

3.3.4 Leiden

Traag *et al.* (2019) introduces the leiden algorithm for community detection. This algorithm adds a refinement phase which guarantees all the formed communities are connected. The paper proves a hierarchy of guarantees.

The algorithm is split into three phases: Local-moving, refinement and aggregation. While the local moving phase is quite similar the leiden algorithm uses ideas of pruning and prioritisation by only considering nodes whose neighbourhood has changed in later iterations. The refinement phase builds the partition that will be aggregated - unlike Louvain the partition is not necessarily the one generated by the local moving phase. There is an essence of randomness with regards to building the partition in the refinement phase. The reason they give for this is that always greedily merging with the community giving highest increase in the quality function may not lead to the optimal partition being found.

There is significant emphasis on comparing Louvain with the leiden algorithm. The percentage of supposedly badly connected communities (communities that when split could improve the quality function) is measured. They show a modularity improvement on all graphs (DBLP, Amazon, IMBD, Live Journal, Web of Science and Web UK) that are benchmarked compared to Louvain. They also report significantly fewer badly connected components.

Takeways include:

- The randomness idea could be applied to our own implementation in the local moving phase slightly changing the Louvain implementation.
- The leiden algorithm is widely considered better than the Louvain algorithm for its ability to reduce the badly connected and internally disconnected communities. The project could further explore this algorithm.
- Currently cuGraph and gLeiden are the major GPU implementations of the leiden algorithm.

3.4 GPU-BASED AND HYBRID STATIC LOUVAIN ALGORITHM

3.4.1 The first GPU accelerated Louvain

Cheong *et al.* (2013) is described as the first parallel community detection algorithm based on the Louvain method. They map the Louvain method onto both single-GPU and multi-GPU architectures.

Their paper proposes separating the parallelism of the algorithm into three levels.

- The first level partitions the network into multiple sub-networks and Louvain is applied in parallel in each of these sub-networks (inter-partition links are removed).
- The second level parallelizes the node visits during modularity optimization phase - nodes are divided into batches for processing. The nodes within a batch are processed in parallel.
- The final level of parallelism is in the computation of modularity gain of moving a node to each of the different neighbouring communities in parallel.

Their profiling for the algorithm run on a graph reveals a key insight - roughly 94.8% of the runtime is spent in the *first pass* of the algorithm, and within that pass about 93% falls in the modularity-optimization phase.

The two dominant kernels of the modularity phase are the *Find Neighboring Communities* (FNC) and *Find Best Move* (FBM) which account for roughly 80% of the computation time.

The graph is stored as an array of structures (node ID, neighbor ID, outgoing and incoming link weights). The FNC is implemented as a sort and reduce pipeline. A kernel first rewrites neighbor IDs to community IDs and prepares sort keys, a Thrust radix sort groups the array by node and then by neighboring community. A second kernel walks each node's contiguous run to accumulate the weights from per (node, community) weights required for modularity gain. FBM assigns a group of threads per node. The third level of parallelism handled across those threads, with the first thread performing the best-community selection.

A note to take is that two nodes processed in the same batch may not be truly independent (one may lie in the other's neighboring community), this race is permitted and relies only on atomic updates to a node's community status.

Their single-GPU version delivered speedups of $1.5\times$ to $5\times$ over the sequential Louvain method but, with the exception of one network, matched or *improved* modularity quality.

Baseline Graphs: uk-2005, webbase-2001, twitter, flickr, livejournal, orkut, and facebook.

Takeaways include:

- It is beneficial to profile and then concentrate of effort in parallelizing the kernels in the first-pass which are the bottlenecks.
- Their parallelization technique - leveling and the kernel implementations for FNC and FBM.

3.4.2 HyDetect

Bhowmik and Vadhiyar (2019) presents HyDetect and Bhowmick *et al.* (2022) extends it to a distributed setting. The former is hybrid CPU-GPU whereas the latter is a multi-node multi-GPU implementation. They propose a novel heuristic to determine *doubtful vertices* which have been assigned to the incorrect communities. Louvain is re-run with the doubtful vertices. This heuristic was implemented as each device invokes Louvain with incomplete information which can cause wrong assignments.

3.4.3 ACLM

Mohammadi *et al.* (2021) (Adaptive CUDA Louvain Method) is a heterogenous CPU-GPU implementation in which the modularity calculation is offloaded to the GPU. This is the GPU-based successor the Fazlali *et al.* (2017) paper.

Their ComputeModularity computes modularity using the modularity formula 2.2. This is run for each edge (u, v) after modifying the graph metadata and generating an

efficient configuration for the GPU computation. The function `ComputeGridConfiguration` in their implementation generates a suitable launch configuration for the `ComputeModularity` function. It determines the number of block per grid, threads per block, shared memory size and number of SMs based on the degree of the vertex v based on the hardware and degree of the moving node.

They baseline against [Staudt and Meyerhenke \(2016\)](#) PLM (NetworKit), APLM [Fazlali et al. \(2017\)](#) and NAIM [Naim et al. \(2017\)](#). ACLM achieves the best execution time on the graph used in the benchmarking (CNR-2000, EU-2005, IN-2004, Cage-15) with near similar modularity.

The main takeaway is the adaptive nature of kernel launch using the compute structure and available hardware.

3.4.4 Nido

[Chou and Ghosh \(2022\)](#) present Nido. They introduce a batched processing style and can process graphs which are larger than the memory present in the GPU. They report performance improvements over shared memory implementations.

3.4.5 cuVite

[Gawande et al. \(2022\)](#) present cuVite which is a distributed multi-GPU C++ library for Louvain community detection which is stated to be the first Louvain implementation using distributed memory on a multi-GPU setup.

The core compute pattern is the local-moving phase being mapped to the GPU while the aggregation is handled by the CPU.

They identify a key bottleneck in porting Louvain to GPUs, that is *determining target communities* for moving a vertex. As we see in (2.3) we need to calculate the sum of weights of edges from a node to each community. Ideally we would want to allocate some sort of map (`community_id` $\rightarrow$ `weight_sum`) for each vertex. They state that they

do not use hash functions deliberately due to the absence of scalable hash functions for the GPU but plan to incorporate them in the future.

The other key bottleneck addressed is load-balancing - for low degree vertices a single thread is allocated whereas for higher degree vertices a cooperative kernel of size 8, 16 or 32 is allocated. They note that the highest performance for cooperative kernels is observed when the CG size is equal to warp size - 32.

cuVite demonstrates upto 2 - 19x improvement over cuGraph on a single node (on observing the table in the paper this seems to be for several large graphs), however it is slower on many graphs.

Baseline graphs: hollywood-2009, nlpkkt240, uk-2002/2005/2007, webbase-2001, com-friendster.

Takeaways include:

- The discussion around how `community_id` $\rightarrow$ `weight_sum` per vertex should be handled.
- Degree based load balancing technique.

3.4.6 cuGraph

NVIDIA Corporation is the github repository for NVIDIA RAPIDS cuGraph which is a popular library for GPU based graph algorithms. It does not have a standalone paper.

3.4.7 ν -Louvain

Sahu (2025) presents performance insights from two implementations GVE-Louvain (multi-core) and ν -Louvain (GPU implementation). They suggest that due to the reduced workload in later Louvain passes ν -Louvain only performs on par with GVE-Louvain.

3.5 CPU-BASED DYNAMIC LOUVAIN ALGORITHM

3.5.1 Shang *et al.* (2014)

This work classifies added edges into four categories and applies operations to the communities based on rules:

- Half-new edge: One of the endpoints is a new node. Either leave the new node in its own community or add it to the community it is connected to based on modularity values.
- New edge: Both endpoints are new nodes. Either add both nodes to some existing community or place both in a new community.
- Inner-Community edge: Both edge endpoints exist and are in the same community. In this case they do not change the community structure as edges added between nodes in the same community should only strengthen the community.
- Cross-Community edge: Both endpoints exist and are in different communities. Decides between merging the communities or changing nothing based on the modularity values.

They evaluate on email-Enron, cit-HepTh, wiki-Vite and FSIM graphs with significant gains in speed as re-clustering is not performed.

Takeaways Include:

- Processing new nodes is a new aspect which is not explored in our work. This can be something we can look into.
- Their work does not handle deletions and merging two communities based on an edge additions seems too drastic. I would imagine a more fine-grained approach would be more beneficial to quality - we do this. That would require more compute, but that is abundant on the GPU.

3.5.2 Cordeiro *et al.* (2016)

This work extends Shang *et al.* (2014) by adding edge deletion support to the incremental algorithm. The framework for edge additions is the same, however *deletions* are handled as follows:

- Edge to an isolated node: Remove the isolated node from its community and place in its own new community.

- Edge between two isolated nodes: Remove the two nodes from whatever community they belong to. This edge deletion is equivalent to deleting the nodes as a fully isolated node does not affect the modularity of a graph.
- Inner-Community edge: Either community structure is not changed or split into two or more communities.
- Cross-Community edge: Community structure is not changed as deleting an edge between communities should improve the modularity of the graph.

An interesting point is that they use three new metrics to evaluate quality: Newmann's modularity, modularity with split penalty and Modularity density.

Quite small networks are used as baselines - cit-Hepth and AS-733 from SNAP.

Takeaways Include:

- Other metrics used for quality evaluation.
- Method for handling edge deletions.

3.5.3 C-Blondel

Seifikar *et al.* (2020) introduces C-blondel which separates graph changes into node addition, node deletion, edge addition and edge deletion. This is different from Shang *et al.* (2014) and Cordeiro *et al.* (2016) which only support Edge-based actions. While node addition and deletion can be emulated by edge-modifications this paper directly addresses node-based modifications as well.

The algorithm processes a graph change in the following manner:

- Remove all affected nodes from their communities by calling a `RemoveNode` function. The resulting graph is called a *Compressed Graph*.
- Call the Louvain algorithm on the compressed graph. This is more efficient than calling Louvain on the entire graph because the compressed graph contains far fewer super-nodes as it is already partially clustered - only affected nodes are removed.

They employ a *Destructive Node Heuristic* for implementing the `RemoveNode` function.

When a node is removed from a community it is possible that the remaining nodes in the

community may not be tightly connected with each other. Therefore, the compressed graph may need to be generated with nodes of communities separated into their own super-nodes. They employ a heuristic to determine when to do this based on the degree of the removed node.

The baseline graphs are Cit-HepTh, email-Enron and a facebook data set of friendships. The baseline algorithms are S-Blondel and D-Blondel.

Takeaways include:

- The destructive node heuristic.
- Direct support for node deletion and addition graph operations.

3.5.4 Naive-Dynamic, Delta Screening & DF-Louvain

Aynaud and Guillaume (2010), Zarayeneh and Kalyanaraman (2021) and Sahu (2024) are the three dynamic algorithms ported to the GPU. The algorithms are discussed further in 4.3

3.6 GPU-BASED DYNAMIC LOUVAIN ALGORITHM

No literature for this section could be found by me.

CHAPTER 4

WORK DETAIL

4.1 UGRC WORK

This project builds on my UGRC-I work *Dynamic Louvain on GPUs with CUDA*, which aimed to accelerate community detection on evolving graphs using NVIDIA GPUs. The following was implemented and evaluated in the UGRC project:

- A **sequential Louvain algorithm on the CPU**, serving as a correct reference implementation and as a means to understand the method in full detail.
- A **simple dynamic Louvain algorithm on the CPU**, which warm-starts from the community assignment of the previous snapshot instead of recomputing the communities from scratch.
- A **static Louvain algorithm for GPUs** written in CUDA, using the Compressed Sparse Row (CSR) graph representation, cooperative-group kernels for grid-wide synchronisation across the iterations of the local-moving phase, and on-device graph aggregation.
- A **benchmark of the GPU static Louvain** against the cuGraph library and against my own sequential CPU implementation.

Several issues were left open at the end of that phase, and motivate the present work:

- **Correctness of the GPU implementation.** The GPU static Louvain was not fully correct: it suffered from synchronisation and correctness problems, the most serious being that on larger graphs the communities would collapse into a single node.
- **Limited evaluation.** The only baselines were the sequential CPU implementation and cuGraph, and testing used only randomly generated graphs rather than real-world networks.
- **Dynamic louvain for GPUs** was not implemented due to the time taken in understanding the louvain algorithm and implementation of the CPU and GPU-static variants. Additionally, we did not have a deep understanding of the dynamic algorithm.

The present work addresses these gaps - correctness of the parallel implementation,

evaluation against established baseline implementations on real-world static and temporal graphs, and a complete set of dynamic variants (naive-dynamic, delta-screening, and dynamic-frontier). A deep literature review of existing work was conducted - both static and dynamic louvain implementations.

4.2 PROJECT TIMELINE

A rough timeline of the work done is presented in Table 4.1.

Period	Summary	Activities
Month 1	Baseline survey and environment setup	Surveyed performance baselines and prior implementations (Vite, NetworkX, cuVite, cuGraph, GALA, Gunrock, and DF-Louvain); ran cuGraph on Google Colab and NetworkX through its Python library; figured out submitting jobs on the Aqua cluster.
Month 2	Literature review and start of implementation	Studied the delta-screening and DF-Louvain papers and carried out a broader literature review; began the CUDA implementation of the dynamic Louvain algorithm.
Month 3	First large-graph testing and bug discovery	Tested on two large graphs (LiveJournal and a German road network); a single pass took around 30 minutes and the output quality on the second graph was poor, exposing correctness and synchronisation bugs to be fixed.
Month 4	Benchmark expansion and correctness fixes	Selected further benchmark graphs (SNAP, SNAP temporal, and randomly generated graphs) and began benchmarking on them; ran cuGraph on the Aqua cluster; fixed a modularity-gain correctness bug.
Month 5	Additional baselines, Final benchmarking and report writing	Continued the literature review, fixed more synchronisation bugs, finalised the implementation, and began writing this report; set up and ran Vite, NetworKit, and Nido alongside cuGraph and NetworkX, finally choosing cuGraph, NetworkX and NetworKit as the baselines.

Table 4.1: Project timeline.

The early phases focused on surveying the baselines and the literature and on bringing up the implementation with the middle phases on testing at scale and fixing the correctness issues this exposed. The final phase focused on broadening the set of baselines and writing this report.

4.3 DYNAMIC LOUVAIN COMMUNITY DETECTION ALGORITHMS

In 2.2 we have seen the normal louvain community detection algorithm. Static community detection algorithms output a community assignment given a graph. Dynamic community detection adds another layer: it involves a static community detection run on an initial graph and a step-wise modification algorithm run for each graph update. The static algorithm establishes the communities of the initial graph, and a dynamic algorithm then updates them after each batch of edge changes, reusing the previous result rather than recomputing the communities from scratch. When only a small part of a large graph changes, recomputing everything wastes most of the effort, so a dynamic algorithm restricts the computation to the region that the change can actually influence.

4.3.1 Dynamic Graphs and Batch Updates

A dynamic graph is modelled as a sequence of snapshots $G^0, G^1, \dots, G^t$, where G^t is the graph at time step t . The change from one snapshot to the next is a *batch update* $\Delta^t = \Delta^{t-} \cup \Delta^{t+}$, where Δ^{t-} is the set of edges deleted and Δ^{t+} is the set of edges inserted between G^{t-1} and G^t . A dynamic algorithm pass takes the previous community assignment C^{t-1} together with the batch Δ^t , and produces the updated assignment C^t .

The two phases of the algorithm - local moving and aggregation - are unchanged from the static case, and the modularity gain of a candidate move is still given by Equation (2.3).

The dynamic algorithms differ from the static one in only two respects:

- **Warm start.** Instead of beginning with every node in its own community, each node is initialised to its community in the previous assignment C^{t-1} . As most of the graph is unchanged, this partition is already close to optimal and only a few moves are needed to reach convergence.

- **Affected vertices.** Only the nodes whose community is likely to change as a result of Δ^t are processed in the first local-moving pass, rather than all nodes. The three variants described below differ only in how this set of *affected* nodes is determined. The later passes, which operate on the much smaller aggregated graph, process every super-node as in the static algorithm.

Identifying the affected set is the central design choice. If it is too small, nodes that should move are missed and the quality of the resulting communities degrades. If it is too large, it may not be advantageous compared to a full re-run.

4.3.2 Naive Dynamic

The naive dynamic approach [Aynaud and Guillaume \(2010\)](#) warm-starts from C^{t-1} but marks *every* node as affected, i.e. it runs the local-moving and aggregation phases over the whole graph with the warm-start communities. It is still usually faster because of the warm start, but it does nothing to exploit the locality of the change.

4.3.3 Delta Screening

Delta screening [Zarayeneh and Kalyanaraman \(2021\)](#) identifies, before the passes begin, an approximate region of the graph that the batch is likely to affect, using the modularity gain to screen which nodes to include. The edges of the batch are first grouped by their source node, and nodes (and entire communities) are marked as affected based on the following high-level rules:

- **Deletions.** For a deleted edge (i, j) whose endpoints lie in the same community ($c_i^{t-1} = c_j^{t-1}$), the node i , all neighbours of i , and every node of the community c_i^{t-1} are marked affected, since removing an intra-community edge can weaken that community. A deletion between two different communities only strengthens their separation, and is ignored.
- **Insertions.** Among the edges inserted at a source node i , the target community c^* that gives the largest modularity gain is selected. If this gain is positive, the node i , all neighbours of i , and every node of community c^* are marked affected. An insertion within a single community is ignored.

Because delta screening marks entire communities, it may over-estimate the affected set, however its screening test prunes far more work than reprocessing the whole graph. The version implemented here is the *parallel* adaptation of delta screening described

by Sahu (2024), which differs from the original sequential formulation: the original’s deferral check - which skips marking from source i when the chosen target vertex has a stronger incentive to move in the opposite direction - is dropped. Omitting this check only enlarges the affected set, so the result remains a safe superset.

4.3.4 Dynamic Frontier

The dynamic frontier approach Sahu (2024) marks a much smaller initial set and then grows it incrementally while the dynamic algorithm runs. The initial marking considers only the endpoints of the changed edges:

- For a deleted edge (i, j) within a single community ($c_i^{t-1} = c_j^{t-1}$), mark i and j .
- For an inserted edge (i, j) between two different communities ($c_i^{t-1} \neq c_j^{t-1}$), mark i and j .

The frontier then expands during the local-moving phase: whenever a node changes its community, all of its neighbours are marked affected, while the node itself is deactivated from further processing unless a neighbour later reactivates it. This continues until no node changes its community. As a result, only the nodes reachable from the actual point of change are processed.

4.4 IMPLEMENTATION

The dynamic algorithm is implemented from scratch. We combine the dynamic variants into one program - one process computes the static clustering and all three dynamic strategies - Naive-Dynamic (ND), Dynamic-Frontier (DF), and Delta-Screening (DS) - through one shared GPU Louvain driver.

4.4.1 Data structures

The device graph is a plain SoA CSR (Listing 4.1). Keeping `off`, `dst`, `w` in separate arrays gives coalesced access in the edge-scanning loops, and storing arcs (not edges) lets a thread read all of a vertex’s neighbours as one contiguous range `[off[v], off[v+1]]`. Weights, degrees (K), community totals (Σ) and the accumulated modularity gain are of

type double.

Best-community selection needs, per vertex, the weight to each neighbouring community as per 2.3. This is held in a *per-vertex* open-addressing hashtable: a single global pair of arrays htKey/htVal partitioned by a prefix-sum offset array htOff, where vertex v owns the slice $[\text{htOff}[v], \text{htOff}[v+1])$. Each slice is sized to the next power of two $\geq \deg(v) + 1$ (Listing 4.1), which guarantees a free slot so linear probing always terminates. The table lives in *global* memory rather than shared memory because its size is per-vertex and a high-degree hub needs a slice far larger than shared-memory could provide.

The remaining per-vertex state is a set of length- N device arrays: the community label C , the degree K , the community totals Sigma , the mapping orig (original vertex $\rightarrow$ current super-vertex), the active/range frontier masks, and a per-community spin-lock commLock .

```
1 // Structure-of-arrays CSR of directed arcs (each undirected edge =
   // two arcs).
2 struct DGraph {
3     int      N = 0;          // vertices
4     int      M = 0;          // arcs (directed) = 2 * |E|
5     int*     off = nullptr;   // size N+1 (CSR row offsets)
6     int*     dst = nullptr;   // size M (neighbour ids)
7     double*  w  = nullptr;   // size M (arc weights)
8 };
9
10 __device__ __forceinline__ unsigned hash_u(int c) {
11     return (unsigned)c * 2654435761u; // multiplicative hash
12 }
13
14 // Per-vertex hashtable capacity = next power of two >= deg+1, so an
   // empty slot
15 // always exists (linear probing terminates) and the index wraps with
   // a bit-mask.
16 __global__ void htCap_kernel(int N, const int* off, int* cap) {
17     int v = blockIdx.x * blockDim.x + threadIdx.x;
18     if (v >= N) return;
19     int need = (off[v + 1] - off[v]) + 1;
20     int c = 1;
```

```

21 while (c < need) c <=< 1;
22 cap[v] = c;
23 }

```

Listing 4.1: Structure-of-arrays CSR and per-vertex hashtable sizing.

4.4.2 The local-moving kernel

The heart of the implementation is the local-moving kernel. It is shown as pseudocode in Listing 4.2; the full CUDA kernel (the per-vertex hashtable inserts, the lock acquisition, and the commit) is given in Listing A.1 in Appendix A.1. Each thread owns one vertex v and proceeds in four steps:

1. **Accumulate.** Clear the vertex's hashtable slice and add the weight from v to each neighbouring community by open-addressing inserts.
2. **Optimistic pick.** Scan the slice and select the community best_c with the largest gain, reading Σ *optimistically* - the values may be stale because other threads are updating them concurrently.
3. **Lock.** If a beneficial target exists, acquire a lock on the *pair* of communities ($d, \text{best_c}$), taking the lower community id first.
4. **Verify and commit.** With both communities locked - and therefore frozen - re-scan the exact K_{id} , K_{ic} , re-read Σ_d , Σ_c , recompute the gain, and commit (update Σ and $C[v]$, re-activate neighbours) only if it is still positive.

```

1 parallel for each vertex v that is active (and allowed by range):
2   mark v processed
3   H <- empty per-vertex hashtable;
4   d <- C[v];
5   k_i <- K[v];
6
7   // (1) accumulate weight from v to each neighbouring community
8   for each neighbour u of v with u != v:
9     H[C[u]] <- H[C[u]] + w(v, u)
10
11  // (2) Find best target, using current (possibly stale) Sigma
12  best_c <- d;
13  best_dQ <- 0;
14  for each community c in H with c != d:
15    dQ <- 2 (H[c] - H[d]) / M - 2 k_i (k_i + Sigma[c] - Sigma[d
16      ]) / M^2
17    if dQ > best_dQ:

```

```

17         best_c <- c;
18         best_dQ <- dQ;
19
20     if best_c = d or best_dQ <= 0:
21         return
22
23     // (3) lock the community pair, lower id first (deadlock-free
24         ordering)
25     lock( min(d, best_c) );
26     lock( max(d, best_c) );
27
28     // (4) re-evaluate with frozen membership
29     // EXACT gain, then commit
30     recompute K_{v->d}, K_{v->best_c}, Sigma[d], Sigma[best_c]
31     dQ2 <- 2 (K_{v->best_c} - K_{v->d}) / M
32             - 2 k_i (k_i + Sigma[best_c] - Sigma[d]) / M^2
33     if dQ2 > 0:
34         Sigma[d] <- Sigma[d] - k_i;
35         Sigma[best_c] <- Sigma[best_c] + k_i
36         C[v] <- best_c
37         dQ_total <- dQ_total + dQ2          // atomic add
38         memory fence
39         re-activate every neighbour of v
40         // DF also adds them to the range
41     unlock( max(d, best_c) ); unlock( min(d, best_c) )

```

Listing 4.2: Local-moving iteration (one thread per vertex)

4.4.3 Aggregation

After a pass converges, communities are contracted into super-vertices (Listing 4.3). Each arc is keyed by the pair of communities it connects, $key[e] = C[src[e]] \cdot nc + C[dst[e]]$, which is unique per ordered community pair. Sorting the arcs by this key with `sort_by_key` (carrying the weights) groups all parallel arcs together, and `reduce_by_key` sums their weights in one pass; the reduced keys are then decoded back into (src, dst) and an exclusive scan of the per-source counts produces the new CSR offsets. Self-loops are retained on purpose - they carry the internal community weight σ_c that the modularity and the coarsened degrees depend on.

```

1 static void aggregate(const DGraph& g, const int* d_C, int nc, DGraph
    & out) {

```

```

2   int M = g.M;
3   int* d_src;  cudaMalloc(&d_src, sizeof(int) * M);
4   writeSrc_kernel<<<nblocks(g.N), TPB>>>(g.N, g.off, d_src);    //
        src[e] = owner
5
6   long long* d_key; cudaMalloc(&d_key, sizeof(long long) * M);
7   buildKey_kernel<<<nblocks(M), TPB>>>(M, d_src, g.dst, d_C,    //
        key = C[src]*nc
8                                   (long long)nc, d_key);    //
                                   + C[dst]
9
10  // sort arcs by key (carrying weights), then sum duplicates
11  thrust::sort_by_key(key, key + M, wt);
12  auto end = thrust::reduce_by_key(key, key + M, wt, okey, ow);
13  int Mr = (int)(end.first - okey);    //
        unique arcs
14
15  // decode reduced keys to (src,dst); counts per src -> CSR
        offsets
16  decode_kernel<<<nblocks(Mr), TPB>>>(Mr, d_okey, (long long)nc,
17                                   d_osrc, out.dst, out.off);
18  thrust::exclusive_scan(offp, offp + nc + 1, offp);    //
        counts -> offsets
19 }

```

Listing 4.3: Aggregation via Thrust `sort_by_key` + `reduce_by_key` on the community-pair key.

4.4.4 The Louvain Driver

The driver (Listing 4.4) is the multi-level Louvain loop shared by the static pass and all three dynamic variants; they differ only in the warm-start labels and the initial active/range masks handed to it. It alternates a local-moving phase with a coarsening (aggregation) phase until the partition stops improving, and tracks the community of every original vertex in the mapping `d_orig`.

```

1  for (int pass = 0; pass < MAX_PASSES; ++pass) {
2      int* range_ptr; // later passes: no range limit
3      if(firstPass) range_ptr = d_range;
4      else range_ptr = nullptr;
5
6      int df_flag = (firstPass && isDF) ? 1 : 0;
7      if(firstPass && isDF) df_flag = 1;
8      else df_flag = 0;

```

```

9
10     cudaMemset(d_commLock, 0, sizeof(int) * g.N); // all locks free
11
12     int iters = 0;
13     for (int it = 0; it < MAX_ITERATIONS; ++it) {
14         iters++;
15         cudaMemset(d_dQ, 0, sizeof(double));
16         localMove_kernel<<<nblocks(g.N), TPB>>>(
17             g.N, g.off, g.dst, g.w, d_C, d_K, d_Sigma,
18             d_active, range_ptr, df_flag,
19             d_htOff, d_htKey, d_htVal, M_total, d_dQ, d_commLock);
20         double hdQ = 0.0;
21         cudaMemcpy(&hdQ, d_dQ, sizeof(double), cudaMemcpyDeviceToHost
22             );
23         if (hdQ <= tau) break; // locally converged
24     }
25
26     // renumber communities to [0,nc)
27     int nc = countCommunities(d_C, g.N, d_present);
28     thrust::inclusive_scan(pp, pp + g.N, pp);
29     fold_kernel    <<<nblocks(N),    TPB>>>(N,    d_orig, d_C,
30         d_present);
31     renumber_kernel<<<nblocks(g.N), TPB>>>(g.N, d_C, d_present);
32
33     // stop, or coarsen and recurse (Alg. 4)
34     if (iters <= 1 || (double)nc / g.N > AGG_TOLERANCE)
35         break;
36     DGraph ng;
37     aggregate(g, d_C, nc, ng);
38     freeDGraph(g);
39     g = ng;
40     /* reset K, Sigma=K, C=identity, active=1 on the coarsened graph
41         ... */
42     tau = tau / TOLERANCE_DROP;
43     firstPass = false;
44 }

```

Listing 4.4: The driver loop

Each outer iteration is a *pass*. A pass first frees all community locks (`cudaMemset(d_commLock, 0)`) and then runs the local-moving phase: it relaunches `localMove_kernel` up to `MAX_ITERATIONS` times, zeroing the gain accumulator `d_dQ` before each launch and copying it back afterwards, and stops once a sweep's total gain

falls below a tolerance `tau` (the partition has locally converged). Only the *first* pass is handed a non-null `range` and the `isDF` flag - the dynamic frontier and the delta-screen restrict work at the finest level only. Coarser passes run an unrestricted local move over the super-graph - which have far few nodes.

When the local-moving phase converges, the labels are compacted: `countCommunities` marks which community ids are in use, an inclusive scan turns that mask into a dense renumbering $[0, nc)$, `fold_kernel` folds this renumbering through into the mapping `d_orig` (so each *original* vertex's community stays current as the levels coarsen), and `renumber_kernel` relabels the working array `C`.

The driver can either choose to stop or coarsen and continue. It stops when the pass produces no productive move or when the community count barely shrank ($nc/N > \text{AGG_TOLERANCE}$). Otherwise it contracts the graph with `aggregate` - one super-vertex per community - and resets the per-vertex state for the smaller graph: super-vertex degrees K , totals $\Sigma = K$ (each super-vertex starts in its own community), identity labels, and all vertices active. It then tightens the tolerance (`tau /= TOLERANCE_DROP`). After the loop terminates, `d_orig` holds the final community of every original vertex, which is copied back to the host.

4.5 MARKING AND THE DYNAMIC VARIANTS

All four entry points run the same driver (Listing 4.4) and differ only in two inputs: the warm-start community labels and the initial `active/range` masks that seed the first local-moving pass.

The variants are distinguished by how they choose what to process. The algorithms are described at 4.3.

- **Warm start.** The three dynamic variants begin from the previous partition C^{t-1} instead of all-singletons, so only a few moves are needed near the change. This is a major benefit as this reduces the majority of the processing.

- **ND (naive-dynamic)** marks *every* vertex as affected: it re-optimises the whole graph from the warm start, exploiting nothing about *where* the change occurred.
- **DF (dynamic-frontier)** marks only the endpoints of the changed edges as affected and then *grows* the frontier on the device - the iSDF branch of the kernel re-activates a moved vertex's neighbours, so the affected region expands outward, following the moves.
- **DS (delta-screening)** computes a fixed affected super-set once (range) and never grows it, confining the cascade of re-activations to that set.

4.6 SYNCHRONISATION CONSIDERATIONS

There are several synchronisation considerations made. We discuss a few of them:

Per-iteration kernel launches (no device-wide barrier). A local-moving pass is not a single persistent kernel. The *host* relaunches the kernel once per iteration and decides convergence based on the accumulated gain (Listing 4.4). The kernel launch boundary *is* the global barrier between iterations, which avoids the need for **cooperative-group grid synchronisation** and keeps the kernel simple.

Community-pair spin-locks. The verified commit holds a lock on *both* affected communities. Locks are plain `int` flags taken with `atomicCAS(&lock, 0, 1)` and released with `atomicExch(&lock, 0)`. To prevent deadlock the two locks are always acquired in increasing community-id order (*lo* then *hi*).

This spin-lock is only deadlock-free under *Independent Thread Scheduling* (Volta and later, compute capability ≥ 7.0); on the RTX 3050 (sm_86) two threads in the same warp can make independent progress so a lock held by one lane does not livelock its warp-mates. This dependence is not merely theoretical.

Compiling the identical source for a pre-Volta target (`nvcc -O3 -arch=sm_60`, Pascal, which lacks Independent Thread Scheduling) and running it on even the tiny karate graph

causes the local-move kernel to *hang* - it does not terminate - whereas the `sm_86` build clusters the same graph in milliseconds. Without Independent Thread Scheduling the lanes of a warp share a single program counter, so a lane spinning on `commLock` starves the warp-mate that holds it, which can therefore never reach the `atomicExch` that would release the lock.

Optimistic pick, verified commit. The two-phase scheme of the local move - an optimistic pick followed by a commit that is verified under the community-pair lock - is what makes it correct. Without it, threads may move vertices based on modularity gains calculated against stale Σ . This can admit modularity-*decreasing* moves and 2-cycle swaps (the thread processing vertex u which is in community C_u is moving u to community C_v while the thread processing vertex v is moving v to community C_u). Because a move only mutates Σ_d , Σ_{best_c} and $C[u]$, holding both community locks means no other thread can change the membership of either community while the gain is verified, so every committed move provably increases modularity.

Atomics and memory fences. Two atomic operations are used: `atomicAdd` accumulates the per-iteration modularity gain into `dQ_accum` (the convergence signal), and `atomicAdd` counts arcs per super-vertex during aggregation. The, the Σ (Sigma in code) updates inside the commit are non-atomic writes: only the two lock holders may touch Σ_d and Σ_{best_c} , so no atomic is required. A `__threadfence()` after the commit publishes the new C/Σ to global memory *before* the locks are released, so the next thread to lock those communities reads the correct values.

Frontier updates. When a vertex moves it re-activates its neighbours with `active[u]=1`. Several moving vertices may write the same `active[u]` concurrently, but every writer stores the same value, so these writes do not need an atomic or lock.

4.7 OTHER CONSIDERATIONS

4.7.1 Vertex-based versus edge-based parallelism

The static GPU implementation built during the UGRC was *edge*-parallel: the arc array was split into equal contiguous blocks, one per active thread, and each thread walked its block proposing, for every arc (u, v) , to move the source u into the community of the destination v . Splitting by arc count balances the work evenly across threads, but making a single move safe was expensive - it acquired *four* spin-locks: both endpoint vertices u and v (so that two arcs sharing a vertex cannot move it at the same time) and both affected communities (so the community totals stay consistent while the gain is evaluated and applied). Because a high-degree vertex is an endpoint of many arcs, its vertex lock and its community's lock become hot spots, and the locking cost grows with degree.

The present implementation is *vertex*-parallel. One thread owns a vertex and scans all of its incident arcs into a thread-private hashtable (the per-vertex table of Listing 4.1), so the per-neighbour accumulation needs *no* synchronisation at all. The cost is load imbalance: the thread that owns a high-degree hub does far more work than its warp-mates and stalls them until it finishes, which is why runtime on heavy-tailed social graphs is poor.

4.7.2 Cooperative kernels and the grid-size limit

To synchronise the iterations of the local-moving phase, the UGRC implementation used a cooperative kernel with grid-wide `grid.sync()` barriers. A cooperative launch requires every block of the grid to be co-resident on the streaming multiprocessors at once, so that all blocks can reach the barrier together. This caps the grid at the device's occupancy - roughly the number of SMs times the blocks each SM can hold. The following is the cooperative kernel launch used in the UGRC for the louvain kernel (one pass) with 32 blocks of 512 threads each.

```
1 cudaLaunchCooperativeKernel((void*)louvain_kernel, 32, 512,  
    kernelArgs);
```

Listing 4.5: cooperative kernel launch

The RTX3050 for my laptop comes with 16 SMs and 1536 threads per SM. The launch should allow upto 48 blocks with 512 threads each (generally). Hence, on modifying this to 64 blocks of 512 threads each and running results in the following error at runtime:

```
bash

$ nvcc -rdc=true -arch=sm_86 static_louvain.cu -o static_louvain
$ ./static_louvain.exe < graph.txt
CUDA error static_louvain.cu:483: too many blocks in cooperative
launch
```

Using our current one thread per vertex, a graph with millions of vertices needs far more than can be co-resident. The present design sidesteps this entirely. As discussed in the synchronisation considerations above, the host relaunches an ordinary (non-cooperative) kernel once per iteration and uses the launch boundary itself as the barrier. An ordinary launch has no co-residency requirement, so the grid is sized to the problem and scales to graphs of any size - the only overhead is the launch itself and more host-device communication.

4.7.3 Convergence thresholds and the move epsilon

The driver exposes a handful of constants that trade quality against runtime.

- **TOLERANCE** (τ) is the per-pass gain threshold: a local-moving pass stops once the total modularity gain of a sweep falls below τ .
- **TOLERANCE_DROP** ($= 10$) makes that threshold level-dependent. τ begins at **TOLERANCE** on the finest graph and is divided by **TOLERANCE_DROP** after each level. Suppose **TOLERANCE_DROP** $= 10$ and initially $\tau = 10^{-2}$ then it follows the sequence - $\tau = 10^{-2}, 10^{-3}, 10^{-4}, \dots$
- **AGG_TOLERANCE** stops the outer loop once a pass shrinks the community count by less than a set fraction.
- **MAX_ITERATIONS** / **MAX_PASSES** are safety bounds on the inner and outer loops.
- **MOVE_EPS** ($= 10^{-12}$) is more subtle, and its absence was a concrete problem in the

UGRC code. It is the *strict* positivity threshold a move’s gain must clear before it is committed. Without it we may move vertices based on floating-point noise, and pairs of vertices oscillate between communities on equal-and-opposite near-zero gains. A single pass may take far longer to settle: in the UGRC implementation some passes would not terminate.

4.7.4 A modularity-gain bug from self-loops

The UGRC static implementation also carried a correctness bug in the modularity-gain computation that had surfaced on large graphs. Listing 4.6 shows the routine: it sums the weight $k_{i_in_old}$ from the moving vertex into its old and new communities by scanning the vertex’s adjacency. The scan does not skip the vertex’s own *self-loop* - the arc whose destination is the vertex itself.

After re-deriving the modularity gain equation which is present in 2.1.1 this subtle bug was noticed.

```

1  double delta_Q = 0.0;
2  double k_i_in_old = 0.0;
3  double k_i_in_new = 0.0;
4  double k_i = d_vertex_weight[node_to_move];
5  double sum_tot_old = community_degree[old_community]
6  double sum_tot_new = community_degree[new_community];
7
8  for(int i = d_csr_node_offset[node_to_move]; i <
   d_csr_node_offset[node_to_move + 1]; i++){
9      // Fix would be
10     // if(d_csr_adj[i].dest == node_to_move) continue;
11     if(community[d_csr_adj[i].dest] == old_community){
12         k_i_in_old += d_csr_adj[i].weight;
13     }
14     if(community[d_csr_adj[i].dest] == new_community){
15         k_i_in_new += d_csr_adj[i].weight;
16     }
17 }
18
19 return 2.0 * (k_i_in_new - k_i_in_old) / m + 2.0 * ((k_i * (
   sum_tot_old - sum_tot_new - k_i)) / (m * m));

```

Listing 4.6: Modularity gain calculation in UGRC

A self-loop’s destination is the vertex itself, which is by definition in the old community,

so its weight is wrongly added to that community's $k_{i_in_old}$. The original input graphs have no self-loops, so the bug will not come up in the first pass. However, after aggregation self-loops will form, so from the second pass onward every super-vertex has a large self-loop and the inflated $k_{i_in_old}$ corrupts every subsequent gain - which, on large graphs, **drove the communities to collapse into a single node**. The present louvain iteration kernel ensures not make the same error.

4.7.5 Best-target moves instead of any improving move

The UGRC local move was greedy per arc. The kernel was edge-parallel, and I took a decision that while processing an arc (u, v) to move u into v 's community as soon as that single move showed a positive gain.

The present kernel instead gathers all of a vertex's neighbouring communities into its hashtable, scores them, and commits only the single best target (Listing 4.2). Each vertex therefore takes the community-pair lock at most once per iteration and only if its best move is beneficial, rather than once per improving arc. This both cuts the number of lock acquisitions sharply - and makes convergence steadier, since a vertex settles into its locally best community in one step instead of possibly being moved multiple times.

4.7.6 Coalesced memory access

Memory coalescing is to have multiple threads of a warp touch contiguous addresses (the same block of 32 words) so the hardware serves them in a single transaction. This drove the choice of a structure-of-arrays (SoA) layout.

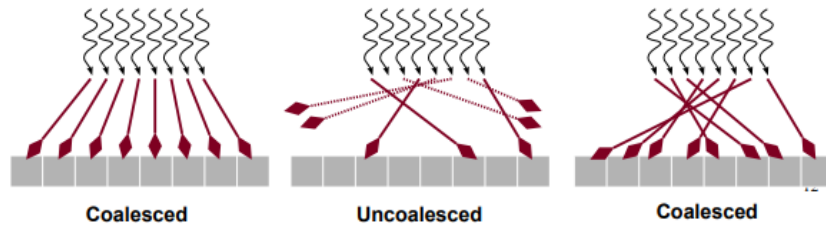


Figure 4.1: adapted from [Nasre](#)

The UGRC graph was an *array of structures*: one `Edge{int src; int dest; double weight;}` array, sixteen bytes per arc. A kernel that reads one field of consecutive arcs (`d_csr_adj[e].dest`) has its lanes land sixteen bytes apart, so only four of every sixteen bytes the hardware fetches are actually used.

The present graph is in a *structure of arrays* format - `off`, `dst`, and `w` are three separate packed arrays (Listing 4.1) - and kernels generally index them by the global thread id, so lane e and lane $e+1$ address adjacent words, coalescing the access. The memory loads/stores from `src`, `dst` and `key` arrays are coalesced in Listing 4.7.

```
1  int thread_id = blockIdx.x * blockDim.x + threadIdx.x;
2  if (thread_id >= M) return;
3  long long a = C[src[thread_id]];
4  long long b = C[dst[thread_id]]
5  key[thread_id] = a * num_communities * b;
```

Listing 4.7: Build key kernel in BTP implementation

CHAPTER 5

BENCHMARKING & RESULTS

This chapter describes the graphs used to evaluate our dynamic Louvain implementation, the pre-processing applied to them, baseline algorithms used and the final results. We compare our GPU implementation against established CPU and GPU baselines (NetworkX, NetworKit, and cuGraph) on three families of graphs: real-world static networks, real-world temporal networks, and synthetically batches on a SNAP graph. Two classic small networks are additionally used to validate correctness against well-known reference partitions.

5.1 GRAPHS

Our real-world graphs are drawn from the Stanford Network Analysis Project (SNAP) [Leskovec and Krevl \(2014\)](https://snap.stanford.edu/data/), available at <https://snap.stanford.edu/data/>. They range from a few thousand to over a million vertices, allowing us to study how each implementation scales. All graphs are treated as undirected and unweighted for community detection. Directed inputs are symmetrised and every edge is assigned unit weight.

5.1.1 Classic Benchmark Graphs

We use two classic small networks from the community-detection literature: Zachary’s karate club (34 vertices, 78 edges) and Lusseau’s dolphin social network (62 vertices, 159 edges). These have well-studied community structure and serve as correctness checks. We expect our implementation to recover modularity values matching the reference baselines rather than as performance benchmarks, since at this scale runtime is likely dominated by fixed GPU overheads.

5.1.2 SNAP Graphs

The static SNAP graphs cover collaboration networks (the ca-* co-authorship graphs), a social ego-network (ego-Facebook), an email communication network (email-Enron), product co-purchasing and co-authorship networks with ground-truth communities (com-Amazon, com-DBLP, com-Youtube), a web hyperlink graph (web-Google) and two large social networks (com-LiveJournal, com-Orkut). Table 5.1 lists them in increasing order of size based on the statistics mentioned on the website [Leskovec and Krevl \(2014\)](#).

Graph	Vertices	Edges	Description
ego-Facebook	4,039	88,234	Social circles from Facebook (anonymized)
ca-GrQc	5,242	14,496	Collaboration network of Arxiv General Relativity
ca-HepTh	9,877	25,998	Collaboration network of Arxiv High Energy Physics Theory
ca-HepPh	12,008	118,521	Collaboration network of Arxiv High Energy Physics
ca-AstroPh	18,772	198,110	Collaboration network of Arxiv Astro Physics
email-Enron	36,692	183,831	Email communication network from Enron
com-Amazon	334,863	925,872	Amazon product network
com-DBLP	317,080	1,049,866	DBLP collaboration network
com-Youtube	1,134,890	2,987,624	Youtube online social network
web-Google	875,713	5,105,039	Web graph from Google
com-LiveJournal	3,997,962	34,681,189	LiveJournal online social network
com-Orkut	3,072,441	117,185,083	Orkut online social network

Table 5.1: Static SNAP benchmark graphs.

Note that some graphs such as ca-HepPh, ca-GrQc, ca-AstroPh, ca-HepTh and email-

Enron are detailed as undirected on the website linked in [Leskovec and Krevl \(2014\)](#) however their SNAP graph file is formatted as a directed graph with both (a, b) and (b, a) edges listed. The header for *email-Enron* is displayed below - website lists 183,831 edges, however 367,662 (exactly double) edges are printed in the graph file *email-Enron.txt*.

```
# Directed graph (each unordered pair of nodes is saved once): Email-Enron.txt
# Enron email network (edge indicated that email was exchanged, undirected edges)
# Nodes: 36692 Edges: 367662
# FromNodeId      ToNodeId
```

5.1.3 SNAP Temporal Graphs

The temporal graphs are interaction streams in which every edge carries a Unix timestamp and the same pair of vertices may appear repeatedly over time. Table 5.2 lists the four temporal graphs we use: a university messaging network (CollegeMsg) and three Stack Exchange question-and-answer interaction networks (sx-*).

Graph	Vertices	Temporal edges	Static edges	Description
CollegeMsg	1,899	59,835	20,296	Messages on a Facebook-like platform at UC-Irvine
sx-mathoverflow	24,818	506,550	239,978	Comments, questions, and answers on Math Overflow
sx-askubuntu	159,316	964,437	596,933	Comments, questions, and answers on Ask Ubuntu
sx-superuser	194,085	1,443,339	924,886	Comments, questions, and answers on Super User

Table 5.2: SNAP temporal benchmark graphs [Leskovec and Krevl \(2014\)](#). “Temporal edges” is the full timestamped stream; “static edges” is the de-duplicated graph.

5.2 CONSISTENT GRAPH FORMAT

Every implementation we benchmark - our implementation, and the NetworkX / NetworKit / cuGraph baselines are constructed to consume a single, common input format, so that all methods run on identical inputs. Any raw dataset is therefore first converted to this format. The format encodes an initial undirected graph followed by an optional sequence of batch updates:

```
n m                # n vertices, m initial undirected edges
u_1 v_1            # the m initial edges, one "u v" per line
...
u_m v_m
B                  # number of batches (0 for a static graph)
d_1 i_1            # batch 1: d_1 deletions, i_1 insertions
... (d_1 deletion edges, one "u v" per line)
... (i_1 insertion edges, one "u v" per line)
...                # repeated for each of the B batches
```

Vertices are integers in the contiguous range $0 \dots n - 1$. Each undirected edge is listed once as a pair $u \ v$. Our dynamic Louvain implementation will add both directions internally and treat every edge as having unit weight. A static graph is simply the special case $B = 0$ (the batch section is empty). This uniform representation lets the same file format both the static and the dynamic experiments.

As a concrete example, a graph on four vertices with four initial edges and a single batch (one deletion and two insertions) is written as:

```
4 4
0 1
0 2
```

```

1 2
2 3
1
1 2
2 3
0 3
1 3

```

The initial graph has edges $(0, 1)$, $(0, 2)$, $(1, 2)$ and $(2, 3)$; the single batch then deletes $(2, 3)$ and inserts $(0, 3)$ and $(1, 3)$. Figure 5.1 depicts this update, with the deleted edge dashed and the inserted edges drawn in bold.

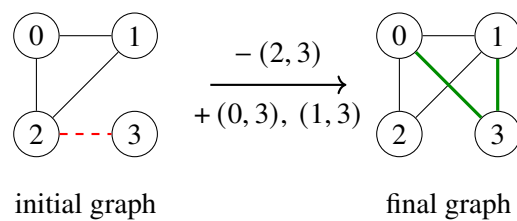


Figure 5.1: The example batch applied to the initial graph.

The subsections below describe the native formats of the SNAP static and temporal graphs and how each is converted to the consistent format.

5.2.1 Static SNAP Graphs Format & Processing

A static SNAP graph is a plain text edge list. It begins with comment lines that start with `#` which recording the graph's name, its vertex and edge counts, and a column header followed by one edge per line as two tab-separated vertex identifiers. The identifiers are arbitrary non-negative integers and need not be contiguous, and an undirected graph is listed with each edge appearing once. A small example is:

```
# Undirected graph: example.txt
```

```
# Nodes: 3 Edges: 3
# FromNodeId    ToNodeId
10      25
25      40
10      40
```

The conversion to our format skips the comment lines and reads each remaining line as an undirected edge; self-loops are dropped and duplicate edges removed; the surviving identifiers are remapped to the contiguous range $0 \dots n - 1$; and the result is written in the consistent format with a trailing batch count of 0 (a static graph). Directed inputs such as web-Google are converted to undirected versions here, each directed arc contributing its undirected edge with duplicates removed. Remapping $10, 25, 40 \mapsto 0, 1, 2$, the example above becomes (The final 0 is the number of batches):

```
3 3
0 1
1 2
0 2
0
```

5.2.2 Temporal SNAP Graphs Format

A temporal SNAP graph is an edge stream with one interaction per line in the form SRC DST UNIXTS. These stand for a source vertex, a destination vertex, and a Unix timestamp. The stream is directed and timestamped, and the same vertex pair typically recurs many times. A small example is:

```
1 2 1082040961
3 2 1082155839
```

```

1 2 1082414391
2 4 1082451031

```

Here the pair (1, 2) appears twice, at different times. The conversion first de-duplicates the stream into a simple undirected graph, keeping for each undirected pair the earliest timestamp at which it appears (self-loops are dropped). In the example this leaves the three undirected edges {1, 2}, {2, 3} and {2, 4}. The unique edges are then sorted chronologically by this first-appearance time and the vertices are remapped to $0 \dots n - 1$. Because a temporal stream only ever adds edges, these batches contain insertions only (no deletions).

5.3 BASELINE ALGORITHMS

We benchmark our implementation against established CPU and GPU Louvain implementations. A range of possible baselines were considered. We list them in Table 5.3 with their type and the outcome.

Name	Type	Status
NetworkX	CPU-static	Used as baseline
NetworKit Staudt and Meyerhenke (2016)	CPU-static (multicore)	Used as baseline
Vite Ghosh et al. (2018b)	CPU-static (MPI + OpenMP)	Successfully run on Colab
Grappolo Lu et al. (2015)	CPU-static (OpenMP)	Available
cuGraph NVIDIA Corporation	GPU-static	Used as baseline
Nido Chou and Ghosh (2022)	GPU-static	Successfully run on Colab
ν -Louvain Sahu (2025)	GPU-static	Available
cuVite Gawande et al. (2022)	GPU-static (multi-GPU)	Did not test
GALA	GPU-static	No implementation found
Gunrock	GPU-static	Did not test
DF-Louvain Sahu (2024)	CPU-dynamic (OpenMP)	Runs but configuration seems incorrect

Table 5.3: Candidate Louvain baselines.

Notably, no available baseline is GPU-*dynamic*. NetworkX, NetworKit and cuGraph are available as python libraries which makes usage straightforward. The code for [Vite](#) and [Nido](#) can be found on GitHub.

5.3.1 NetworkX

A code snippet below clusters a small test graph of two triangles joined by a single edge.

```
1 import networkx as nx
2 from networkx.algorithms.community import louvain_communities,
  modularity
3
4 # two triangles {0,1,2} and {3,4,5} joined by the edge (0,3)
5 G = nx.Graph()
6 G.add_edges_from([(0, 1), (1, 2), (2, 0),
7                  (3, 4), (4, 5), (5, 3),
8                  (0, 3)])
9
10 communities = louvain_communities(G, weight="weight", seed=1)
11 Q = modularity(G, communities, weight="weight")
12 print(len(communities), Q)          # -> 2 communities, Q ~ 0.36
```

Listing 5.1: NetworkX Louvain on a small test graph.

5.3.2 NetworKit

NetworKit exposes the *Parallel Louvain Method* (PLM), which runs multi-threaded on the CPU through OpenMP. We pass `refine=False` for plain (unrefined) Louvain, matching the other baselines.

```
1 import networkit as nk
2
3 G = nk.Graph(6)
4 for u, v in [(0, 1), (1, 2), (2, 0),
5             (3, 4), (4, 5), (5, 3),
6             (0, 3)]:
7     G.addEdge(u, v)
8
9 nk.setSeed(1, True)
10 plm = nk.community.PLM(G, refine=False) # plain (unrefined) Louvain
11 plm.run()
12 part = plm.getPartition()
13 Q = nk.community.Modularity().getQuality(part, G)
14 print(part.numberOfSubsets(), Q)        # -> 2 communities, Q ~ 0.36
```

Listing 5.2: NetworKit PLM (Louvain) on the same test graph.

5.3.3 cuGraph

cuGraph is NVIDIA's RAPIDS GPU graph library. Its Louvain runs entirely on the GPU.

```
1 import cudf, cudgraph
2 import cupy as cp
3
4 edges = [(0, 1), (1, 2), (2, 0),
5          (3, 4), (4, 5), (5, 3),
6          (0, 3)]
7 gdf = cudf.DataFrame({
8     "src":    [u for u, v in edges],
9     "dst":    [v for u, v in edges],
10    "weight": cp.ones(len(edges), dtype="float32"),
11 })
12 G = cudgraph.Graph()
13 G.from_cudf_edgelist(gdf, source="src", destination="dst",
14                      edge_attr="weight", renumber=True)
15
16 parts, Q = cudgraph.louvain(G, resolution=1.0)
17 ncomm = int(parts["partition"].nunique())
18 print(ncomm, Q)                                # -> 2 communities, Q ~ 0.36
```

Listing 5.3: cuGraph (GPU) Louvain on the same test graph.

5.4 ENVIRONMENT

The final experiments are conducted on the google collab platform with the following specifications.

The CUDA code file (*.cu) is compiled with `nvcc -O3 -arch=sm_75 louvain.cu`.

Component	Specification
Platform	Google Colaboratory
CPU	Intel(R) Xeon(R) CPU @ 2.00GHz
Physical cores	1
Logical Processors	2
System RAM	12.7GB
Operating System	Ubuntu 22.04.5 LTS
Hardware Accelerator	NVIDIA T4 GPU
GPU RAM	15.0GB
GPU Driver Version	580.82.07
Driver CUDA Version	13.0
nvcc CUDA toolkit version	release 12.8, V12.8.93
C++ Compiler (Sequential)	g++ 11.4.0
Python Version	3.12.13

Table 5.4: Experimental Environment and Software Versions

5.5 STATIC LOUVAIN BENCHMARKING

5.5.1 Classic Graph Sanity Check

This section presents the results on the two classic benchmark graphs, Zachary’s karate club and Lusseau’s dolphin network. We run our GPU static Louvain alongside the three baselines - cuGraph on the GPU, and NetworkX and NetworKit on the CPU - all executed on Google Colab. Table 5.5 reports, for each implementation, the wall-clock time of a single clustering, the modularity of the resulting partition, and the number of communities detected.

The **partitions agree closely**, which validates **correctness**: on karate our implementation, NetworKit, and cuGraph all reach a modularity of 0.4156 with four communities, while NetworkX settles in a slightly poorer local optimum (0.3854, three communities); on dolphins the four results lie within much closer (modularity 0.516–0.524, five or six communities).

The **runtimes**, however, are dominated by fixed overheads (the memory allocations and other overheads) rather than the negligible clustering work: the CPU baselines finish in well under five milliseconds, whereas the GPU implementations take hundreds of

milliseconds owing to GPU context initialisation and kernel-launch costs. At this scale the classic graphs therefore serve only as correctness checks; the GPU implementations are expected to pay off on the far larger graphs.

Graph	Implementation	Time	Modularity	Comm.
karate	GPU Static Louvain	191.0ms	0.415598	4
	cuGraph	38.9ms	0.415598	4
	NetworkX	4.4ms	0.385355	3
	NetworKit	0.3ms	0.415598	4
dolphins	GPU Static Louvain	211.0ms	0.515921	6
	cuGraph	49.8ms	0.524109	5
	NetworkX	4.0ms	0.518828	5
	NetworKit	0.3ms	0.518828	5

Table 5.5: Performance on the classic benchmark graphs.

Ground Truth	Computed Community	Node	Ground Truth	Computed Community	Node
Instructor	1	0	Instructor	1	17
Instructor	1	1	Officer	3	18
Instructor	1	2	Instructor	1	19
Instructor	1	3	Officer	3	20
Instructor	0	4	Instructor	1	21
Instructor	0	5	Officer	3	22
Instructor	0	6	Officer	3	23
Instructor	1	7	Officer	2	24
Instructor	3	8	Officer	2	25
Officer	1	9	Officer	3	26
Instructor	0	10	Officer	3	27
Instructor	1	11	Officer	2	28
Instructor	1	12	Officer	3	29
Instructor	1	13	Officer	3	30
Officer	3	14	Officer	2	31
Officer	3	15	Officer	3	32
Instructor	0	16	Officer	3	33

Table 5.6: Per-node comparison on Zachary’s karate club.

We compared the **ground-truth factions** (Instructor or Officer) in the Zachary’s karate

club graph versus the community our GPU static Louvain computes in Table 5.6. Blue shades mark the Instructor faction which correspond to communities $\{0, 1\}$ from our algorithm run and orange the Officer faction and corresponding to $\{2, 3\}$ from our run. Only nodes 8 and 9 land on the opposite side (known to be on the boundary). This further validates correctness of our implementation.

5.5.2 Performance on SNAP Graphs

The table below reports the modularity, number of communities, and single-clustering runtime of our GPU static Louvain and the three baselines (cuGraph, NetworkX, and NetworKit) on the first ten static SNAP graphs mentioned in Table 5.1, in increasing order of vertices. Figure 5.2 compares the runtimes of the three fastest implementations - NetworkX is omitted from the figure as it is quite slow.

Communities and Modularity: All four implementations achieve essentially the same modularity on each graph. This agreement among independent implementations confirms that our GPU Louvain recovers correct, reference-quality communities. The number of communities varies a bit more. This is the most clear in com-DBLP: our partitions are among the coarsest (167 communities against cuGraph’s 326). However, it does not stray too far - is in the same order as the baselines for all the graphs. As is characteristic of modularity maximisation, these structurally different partitions nonetheless score almost identically.

Runtime: The runtimes differ far more dramatically. NetworkX, the single-threaded Python baseline, is usable only on the smaller graphs and becomes impractical at scale, needing 69-241 s on the four largest graphs - which is why it is excluded from Figure 5.2, where it would compress every other bar onto the axis. NetworKit’s parallel PLM is in fact the fastest method on the small collaboration networks (just 6 ms on ca-GrQc), where the GPU implementations are dominated by fixed device-initialisation overhead. However it too slows sharply as the graphs grow. Our GPU static Louvain runs in

Graph	Implementation	Time	Modularity	Comm.
ego-Facebook V = 4,039 E = 88,234	GPU Static Louvain	0.240s	0.834932	16
	cuGraph	0.212s	0.834822	14
	NetworkX	1.078s	0.834731	14
	NetworKit	0.143s	0.834975	16
ca-GrQc V = 5,241 E = 14,484	GPU Static Louvain	0.241s	0.859242	389
	cuGraph	0.122s	0.860330	412
	NetworkX	0.378s	0.861277	388
	NetworKit	0.006s	0.862618	390
ca-HepTh V = 9,875 E = 25,973	GPU Static Louvain	0.194s	0.755081	471
	cuGraph	0.123s	0.762040	513
	NetworkX	1.576s	0.766855	482
	NetworKit	0.016s	0.769179	478
ca-HepPh V = 12,006 E = 118,489	GPU Static Louvain	0.206s	0.659764	307
	cuGraph	0.163s	0.657619	327
	NetworkX	2.035s	0.658071	312
	NetworKit	0.026s	0.659710	312
ca-AstroPh V = 18,771 E = 198,050	GPU Static Louvain	0.208s	0.622611	322
	cuGraph	0.368s	0.619396	488
	NetworkX	4.653s	0.625448	328
	NetworKit	0.083s	0.627398	325
email-Enron V = 36,692 E = 183,831	GPU Static Louvain	0.240s	0.606764	1197
	cuGraph	0.438s	0.618807	1279
	NetworkX	5.631s	0.612188	1219
	NetworKit	0.113s	0.597591	1330
com-Amazon V = 334,863 E = 925,872	GPU Static Louvain	0.242s	0.925497	220
	cuGraph	0.763s	0.924840	311
	NetworkX	68.687s	0.925850	238
	NetworKit	1.309s	0.926542	244
com-DBLP V = 317,080 E = 1,049,866	GPU Static Louvain	0.397s	0.818992	167
	cuGraph	1.452s	0.820237	326
	NetworkX	99.229s	0.822063	221
	NetworKit	1.787s	0.822648	257
com-Youtube V = 1,134,890 E = 2,987,624	GPU Static Louvain	3.223s	0.721861	5490
	cuGraph	3.447s	0.723648	5102
	NetworkX	205.433s	0.716040	7317
	NetworKit	4.003s	0.713686	12 283
web-Google V = 875,713 E = 4,322,051	GPU Static Louvain	0.532s	0.977379	3132
	cuGraph	2.154s	0.977017	3448
	NetworkX	240.607s	0.977691	3148
	NetworKit	8.143s	0.977667	3168

Table 5.7: SNAP benchmark graphs in increasing edge-count order. **Note** graph vertex and edge count are listed **post-processing** using the converter mentioned in [5.2](#)

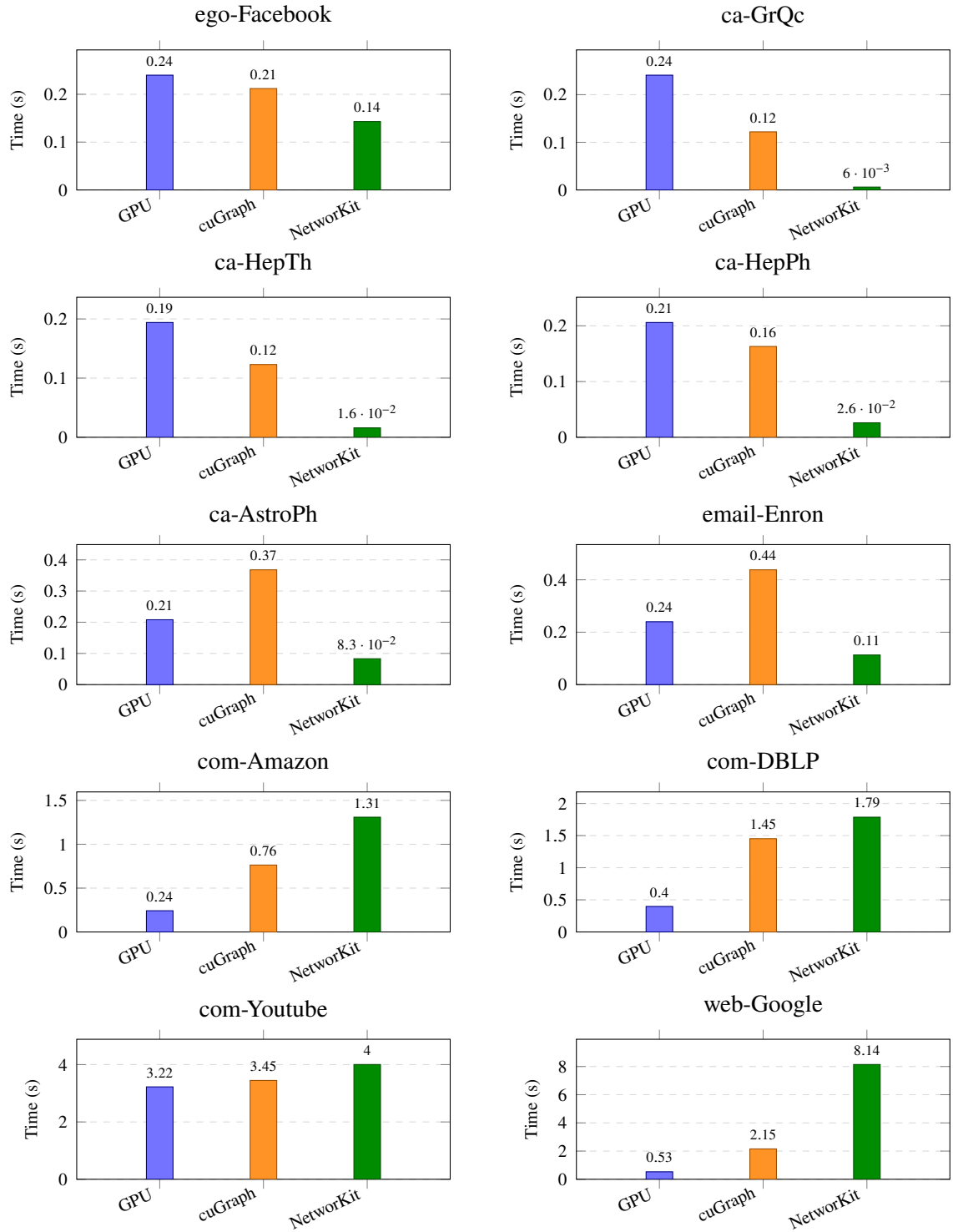


Figure 5.2: Per-graph wall-clock time (seconds) of GPU Static Louvain, cuGraph, and NetworkKit on the SNAP benchmark graphs. Each panel has an independent y-axis. NetworkX is omitted, as its runtimes would dominate the scale.

only a few hundred milliseconds almost independently of graph size, and is the fastest implementation on every large graph: on web-Google it finishes in 0.53 s, roughly 450× faster than NetworkX, 15× faster than NetworKit, and 4× faster than cuGraph.

com-Youtube is interesting as it is difficult for every method. Each method finishes with thousands of small communities. GPU static Louvain is only 3.2 s: is ahead but only marginally compared to cuGraph and NetworKit.

Conclusion: In short, our GPU static Louvain implementation beats all the baselines as the graphs scale into millions of vertices and edges.

5.5.3 Performance on large SNAP Graphs

We additionally benchmark on two of the largest large social-network graphs in the SNAP collection [Leskovec and Krevl \(2014\)](#): com-LiveJournal, and com-Orkut which are also reported in the Table 5.1.

Component	Specification
Platform	Google Colaboratory
CPU	Intel(R) Xeon(R) CPU @ 2.20GHz
Physical cores	6
Logical Processors	12
System RAM	83.5GB
Operating System	Ubuntu 22.04.5 LTS
Hardware Accelerator	NVIDIA A100 GPU
GPU RAM	40.0GB
GPU Driver Version	580.82.07
Driver CUDA Version	13.0
nvcc CUDA toolkit version	release 12.8, V12.8.93
C++ Compiler (Sequential)	g++ 11.4.0
Python Version	3.12.13

Table 5.8: Experimental Environment and Software Versions

While, com-LiveJournal runs to completion on the T4 GPU provided on google collab, it was observed that cuGraph and our GPU static Louvain implementation run out of GPU memory while processing com-Orkut. We therefore run com-LiveJournal and

com-Orkut on an NVIDIA A100, whose larger device memory accommodates the graph. The A100 environment is detailed below in Table 5.8

The CUDA code file (*.cu) is compiled with `nvcc -O3 -arch=sm_80 louvain.cu`.

Table 5.9 reports the modularity, number of communities, and single-clustering runtime of our GPU static Louvain together with the cuGraph and NetworKit baselines on the two graphs. NetworkX is omitted, as a single-threaded run at this scale is impractical.

Table 5.9: Static Louvain on the large SNAP graphs (executed on the NVIDIA A100).

Graph	Implementation	Time	Modularity	Comm.
com-LiveJournal	GPU Static Louvain	27.461s	0.745268	1542
	cuGraph	28.224s	0.734215	17 875
	NetworKit	22.324s	0.743150	2920
com-Orkut	GPU Static Louvain	126.484s	0.669307	30
	cuGraph	67.898s	0.661977	38
	NetworKit	37.081s	0.674177	26

Quality: All three implementations produce partitions of comparable quality on both graphs: the modularities differ by at most 0.012. Our GPU static Louvain attains the highest modularity on com-LiveJournal (0.745) and lies within 0.005 of the best on com-Orkut, confirming that it recovers reference-quality communities even at tens of millions of edges.

Community structure: The two graphs resolve very differently. com-LiveJournal breaks into thousands of communities (1,542 for our method, 2,920 for NetworKit, and 17,875 for cuGraph), whereas com-Orkut - although far larger - collapses into only a few dozen (26–38 across all three methods). This follows from density: com-Orkut has an average degree of ≈ 76 against ≈ 17 for com-LiveJournal, so its vertices agglomerate into a handful of very large communities. On com-LiveJournal, cuGraph produces an order of magnitude more (and smaller) communities than the other two while scoring a slightly *lower* modularity, i.e. it over-fragments the graph; our method instead finds the

coarsest partition at the highest modularity.

Runtime: NetworKit’s parallel PLM is the fastest method on both graphs (22.3 s and 37.1 s). The high number of cores in this environment (12 logical processors, up from 2 logical processors) may be contributing to NetworKit’s performance. Our implementation is competitive on com-LiveJournal (27.5 s, between the two baselines) but is the slowest on com-Orkut (126.5 s, versus 67.9 s for cuGraph and 37.1 s for NetworKit). This gap likely reflects com-Orkut’s **high density** - which inflates the per-vertex cost of our hash-table-based local move and the contention on the commit locks.

5.5.4 Graph Structure Analysis

We observe that relative performance of our GPU static Louvain depends strongly on the *degree distribution* of the input graph rather than on its size alone. Table 5.10 reports degree statistics of the benchmark graphs, measured on the **converted** graphs produced by our preprocessing. Two regimes emerge: com-dblp, com-amazon and web-Google, and the large online social networks: com-Youtube, com-LiveJournal, com-Orkut which have very large maximum degrees.

Table 5.10: Degree statistics of the **converted (pre-processed)** benchmark graphs together with the runtime speedup ratio of our method over the cuGraph and NetworKit baselines (baseline runtime $\div$ our runtime; a value below 1 means we are slower).

Graph	$ V $	$ E $	Avg. deg.	Max. deg.	vs cuGraph	vs NetworKit
com-dblp	317,080	1,049,866	6.6	343	3.66	4.50
com-amazon	334,863	925,872	5.5	549	3.15	5.41
web-Google	875,713	4,322,051	9.9	6,332	4.05	15.31
com-Youtube	1,134,890	2,987,624	5.3	28,754	1.07	1.24
com-LiveJournal	3,997,962	34,681,189	17.4	14,815	1.03	0.81
com-Orkut	3,072,441	117,185,083	76.3	33,313	0.54	0.29

The split tracks maximum degree. On the first three graphs our implementation is 3–4 $\times$ faster than cuGraph and 4–15 $\times$ faster than NetworKit, the strongest baseline (last two columns of Table 5.10). On the social networks, whose maximum degree is one to

two orders of magnitude larger (14,815 for com-LiveJournal, 28,754 for com-Youtube, 33,313 for com-Orkut), that advantage collapses: we are only marginally faster than cuGraph on com-Youtube and com-LiveJournal, and on com-Orkut we are quite *slower* than both cuGraph (0.54 $\times$) and NetworKit (0.29 $\times$). com-Youtube is *hub-concentrated* - its maximum degree (28,754) is over 5,000 $\times$ its average (5.3), whereas com-Orkut, despite being the densest, has a far milder degree skew (maximum only $\approx 440\times$ the average), so its difficulty stems from sheer density rather than from a few dominant hubs.

Why maximum degree matters (load imbalance). Our local-move kernel is *vertex-parallel*: one GPU thread is assigned to each vertex and performs work proportional to that vertex’s degree - it scans the neighbours into a per-vertex hash table and then re-scans the table to select the best community. GPU threads execute in lock-step warps of 32, so a warp cannot retire until its slowest lane finishes. When 31 lanes hold low-degree vertices (a handful of operations each) while one lane holds a hub of degree 28,754, the whole warp stalls for tens of thousands of steps with those lanes idle. A power-law graph has many such hub, so many warps are dragged out and the device is badly under-utilised. In graphs with having very few such outliers, every lane does comparable work and the simple *one thread per vertex* scheme is highly efficient. This is a well-documented failure mode of vertex-centric GPU graph kernels on scale-free graphs; the standard remedy - assigning threads to a vertex in proportion to its degree - is not implemented in our kernel.

Why average degree (density) matters too. com-Orkut is not merely hub-heavy but also very dense (average degree 76, versus 5–17 for the others), and two further costs scale with density. First, each per-vertex hash table is sized to the vertex’s degree, so the aggregate hash-table memory grows with the average degree - this is why com-Orkut exhausts the 16 GB (actually the resources tab in google colab mentions 15) of a T4 GPU and must be run on the A100. Second, the verified-commit step locks the community pair a vertex moves into; com-Orkut collapses into very few communities, so almost

every vertex contends for one of just a few community locks.. com-Orkut therefore has all costs - hub load imbalance, hash-table memory, and lock contention.

Quality is unaffected. These are *efficiency* effects, not correctness ones: the modularity our method attains remains competitive on every graph, and is in fact the highest of all implementations on com-LiveJournal. The performance gap on social graphs is thus a load-balancing limitation of the current kernel rather than an algorithmic weakness, and motivates degree-aware work assignment.

5.6 DYNAMIC LOUVAIN BENCHMARKING

5.6.1 SNAP Temporal Experiment (80% static, $5 \times 4\%$ batches)

This section reports the per-batch behaviour of the four implementations on the four SNAP temporal graphs. The temporal stream of each graph is split into an initial graph and five insertion batches. Our dynamic Louvain implementations process each batch *incrementally* with three strategies namely Naive-Dynamic (ND), Dynamic-Frontier (DF), and Delta-Screening (DS). The existing baselines are static Louvain implementations (cuGraph, NetworKit, and NetworkX) and have no incremental mode. They instead re-cluster the whole graph after every batch. For each graph we report three metrics (modularity, number of communities, and runtime) in a separate table. The first row, *Static pass*, is the initial static clustering, which is identical for ND, DF, and DS.

Partitioning the edges: The unique, time-ordered edges of each temporal graph are partitioned into an initial graph and five update batches. The earliest 80% of the edges form the initial graph, on which the static clustering is computed; the remaining 20% are split evenly, preserving chronological order, into five batches that are applied one after another. Because the temporal stream only adds edges, every batch consists solely of edge insertions.

CollegeMsg

After pre-processing, the graph contains 1,899 vertices and 11,070 initial edges; each of the five insertion batches adds 553–554 edges (2,768 in total).

Batch	ND	DF	DS	cuGraph	NetworKit	NetworkX
Static pass	0.244729	0.244729	0.244729	0.249983	0.261292	0.256886
1	0.256009	0.249111	0.256260	0.251910	0.253242	0.249949
2	0.256073	0.246692	0.256140	0.258523	0.251407	0.256179
3	0.252989	0.241933	0.252654	0.252326	0.260806	0.251037
4	0.250366	0.238308	0.249967	0.237260	0.258796	0.256264
5	0.250191	0.237185	0.249306	0.246782	0.250339	0.264183

Table 5.11: Modularity per batch on CollegeMsg.

Batch	ND	DF	DS	cuGraph	NetworKit	NetworkX
Static Pass	290	290	290	306	290	289
1	235	236	235	235	237	235
2	186	188	184	181	180	179
3	153	157	151	145	148	146
4	100	109	99	111	93	94
5	27	46	27	15	12	13

Table 5.12: Communities per batch on CollegeMsg.

Batch	ND	DF	DS	cuGraph	NetworKit	NetworkX
Static Pass	0.173	0.173	0.173	0.087	0.004	0.314
1	0.002	0.001	0.002	0.059	0.003	0.268
2	0.001	0.000	0.001	0.066	0.004	0.312
3	0.001	0.000	0.001	0.090	0.003	0.342
4	0.001	0.000	0.001	0.051	0.003	0.310
5	0.001	0.001	0.001	0.058	0.004	0.330

Table 5.13: Time (s) per batch on CollegeMsg.

Analysis and Observations:

- DF quality drops by the final batch - the modularity score is consistently lower in most batches.
- All dynamic variants undercoarsen in the final batch - around double the baselines. They all stay near the baselines in terms of number of communities until batch 4; ND and DS stay near baselines in terms of modularity until batch 4 as well.
- Batches 2–4 of DF are reported to be 0.000 seconds. This is because the processing takes less than 1 millisecond which is the granularity that our code reports the time.
- DF is faster than both ND and DS but suffers in quality.

sx-mathoverflow

After pre-processing, the graph contains 24,759 vertices and 150,388 initial edges; each of the five insertion batches adds 7,519–7,520 edges (37,598 in total).

Batch	ND	DF	DS	cuGraph	NetworKit	NetworkX
Static Pass	0.264644	0.264644	0.264644	0.297231	0.301230	0.307303
1	0.313253	0.316536	0.313478	0.310953	0.302251	0.311524
2	0.319826	0.320175	0.320364	0.314642	0.291880	0.298593
3	0.323005	0.321563	0.323936	0.311210	0.296561	0.304751
4	0.326766	0.323090	0.325809	0.309626	0.304076	0.315094
5	0.329280	0.325143	0.328447	0.301103	0.306474	0.309705

Table 5.14: Modularity per batch on sx-mathoverflow.

Batch	ND	DF	DS	cuGraph	NetworKit	NetworkX
Static Pass	5761	5761	5761	5811	5776	5783
1	4622	4622	4622	4637	4646	4634
2	3499	3498	3499	3502	3518	3521
3	2326	2412	2324	2331	2342	2328
4	1230	1390	1305	1242	1233	1234
5	134	289	191	75	60	81

Table 5.15: Communities per batch on sx-mathoverflow.

Batch	ND	DF	DS	cuGraph	NetworKit	NetworkX
Static Pass	0.258	0.258	0.258	0.409	0.052	7.032
1	0.170	0.078	0.174	0.259	0.071	7.580
2	0.032	0.013	0.032	0.253	0.071	6.221
3	0.015	0.010	0.016	0.242	0.066	5.542
4	0.017	0.008	0.013	0.272	0.059	5.242
5	0.016	0.009	0.013	0.297	0.099	6.294

Table 5.16: Time (s) per batch on sx-mathoverflow.

Analysis and Observations:

- Dynamic variants all stay near the baselines in terms of number of communities until batch 4. In the final batch, they end up with several multiples the number of communities as the baselines.
- Nevertheless, the modularity performance of the dynamic variants is better than baselines by around 0.02 by the end of the final batch.
- DF is faster than both ND and DS by around 2×.

sx-askubuntu

After pre-processing, the graph contains 157,222 vertices and 364,552 initial edges; each of the five insertion batches adds 18,227–18,228 edges (91,139 in total).

Batch	ND	DF	DS	cuGraph	NetworKit	NetworkX
Static Pass	0.458727	0.458727	0.458727	0.459580	0.464929	0.460824
1	0.472352	0.458808	0.472319	0.459619	0.466291	0.459881
2	0.471559	0.457983	0.471496	0.451502	0.469667	0.465101
3	0.470583	0.457214	0.470652	0.454922	0.466522	0.465677
4	0.468503	0.455459	0.426168	0.462328	0.466944	0.464869
5	0.310385	0.453776	0.439206	0.463079	0.472183	0.467651

Table 5.17: Modularity per batch on sx-askubuntu.

Batch	ND	DF	DS	cuGraph	NetworKit	NetworkX
Static Pass	33342	33342	33342	33168	33272	33132
1	27384	27380	27387	27299	27429	27251
2	21242	21240	21244	23168	21287	21164
3	14922	14939	14925	15039	14861	14830
4	8746	8747	8732	8901	8665	8638
5	2305	2364	2673	2711	2403	2273

Table 5.18: Communities per batch on sx-askubuntu.

Batch	ND	DF	DS	cuGraph	NetworKit	NetworkX
Static Pass	0.463	0.463	0.463	0.543	0.297	30.663
1	0.136	0.034	0.136	0.683	0.249	32.318
2	0.049	0.061	0.050	0.289	0.277	33.489
3	0.049	0.044	0.055	0.373	0.282	33.701
4	0.050	0.059	0.050	0.430	0.267	31.883
5	0.059	0.051	0.255	0.421	0.367	30.992

Table 5.19: Time (s) per batch on sx-askubuntu.

Analysis and Observations:

- Dynamic algorithms modularity quality drops by the final batch with the most clear being ND becoming 0.31 compared to the baselines around 0.46.
- Community count of all dynamic variants and baselines are similar in all batches.
- The speed of DF is similar in all batches apart from batch 1.

sx-superuser

After pre-processing, the graph contains 192,409 vertices and 571,656 initial edges; each of the five insertion batches adds 28,582–28,583 edges (142,914 in total).

Batch	ND	DF	DS	cuGraph	NetworKit	NetworkX
Static Pass	0.400540	0.400540	0.400540	0.407287	0.412074	0.411821
1	0.417025	0.403317	0.416926	0.410515	0.414086	0.410834
2	0.417341	0.404723	0.417373	0.414947	0.414088	0.414697
3	0.417150	0.405408	0.417242	0.417042	0.417482	0.417306
4	0.416755	0.405830	0.416780	0.412660	0.415204	0.416345
5	0.416187	0.405871	0.416220	0.420041	0.418416	0.417633

Table 5.20: Modularity per batch on sx-superuser.

Batch	ND	DF	DS	cuGraph	NetworKit	NetworkX
Static Pass	46439	46439	46439	46437	46542	46488
1	37404	37406	37408	37400	37522	37363
2	28783	28775	28782	28834	28828	28782
3	19694	19711	19692	19617	19754	19710
4	10631	10627	10636	12503	10639	10661
5	1756	1862	1759	1838	1757	1833

Table 5.21: Communities per batch on sx-superuser.

Batch	ND	DF	DS	cuGraph	NetworKit	NetworkX
Static Pass	0.723	0.723	0.723	0.831	0.403	40.137
1	0.190	0.064	0.194	0.593	0.503	47.978
2	0.070	0.066	0.070	0.640	0.438	45.036
3	0.069	0.068	0.074	0.714	0.407	46.331
4	0.075	0.069	0.069	0.687	0.348	41.236
5	0.074	0.065	0.074	0.777	0.475	49.409

Table 5.22: Time (s) per batch on sx-superuser.

Analysis and Observations:

- The DF variant falls behind DS, ND and other baselines in improving the modularity in the first batch. It also does not improve above 0.41 in any subsequent batch while the other implementation maintain a modularity above 0.41.
- Community count of all dynamic variants and baselines are similar in all batches.
- The speed of DF is similar to ND and DS in all batches apart from batch 1.

Analysis of all SNAP Temporal Experiments

The speed advantage: The dynamic variants are given a warm start and move vertices selectively. The gap between the dynamic variant and baselines spans a few orders of magnitude as seen in Table 5.23. DF is on average *hundreds* of times faster than single-threaded NetworkX, roughly 10× faster than GPU cuGraph (apart from CollegeMsg where it is more than 100× faster), and 3 – 10× faster than NetworKit’s parallel PLM.

Disclaimer: The batch size is just 4% of the total edges. The batch size can also contribute to the speed of the dynamic variants as they only have to process a chosen set of affected vertices as compared to the baselines which are rerun every batch. Increasing the batch size may result in increase in the (dynamic variant per batch time ÷ baseline per batch time) ratio.

Graph	ND	DF	DS	cuGraph	NetworKit	NetworkX
CollegeMsg	1.2	0.4	1.2	64.8	3.3	312.5
sx-mathoverflow	50.0	23.6	49.6	264.5	73.3	6,176.0
sx-askubuntu	68.6	49.8	109.2	439.3	288.6	32,476.4
sx-superuser	95.6	66.4	96.2	682.0	434.4	45,998.0

Table 5.23: Average per-batch update time (milliseconds): the mean over the five insertion batches, excluding the initial static clustering.

Dynamic Frontier (DF) speed: DF is clearly faster than ND/DS on CollegeMsg and sx-mathoverflow while being marginally faster on sx-superuser; It shows nearly equivalent speed on sx-askubuntu apart from the first batch (where it is faster).

Quality sanity check passes and sometimes beats baselines Across all four temporal graphs our incremental variants recover partitions of reference quality (modularity): ND and DS stay within 0.01 of the best static baseline on CollegeMsg and sx-superuser, and *exceed* every baseline by 0.02–0.03 on sx-mathoverflow.

In general the community-count trajectories are nearly identical across all dynamic implementations compared to baselines - confirming that the incremental partition does not drift away from what a from-scratch re-clustering would produce. While we see a slight deviation in a two of the graphs for the final batch, the general trend of reducing communities is still followed and the order of magnitude of the number of communities is still correct.

5.6.2 Quality Degradation Experiments

To probe how the modularity of dynamic incremental methods graph churn, we apply synthetic random batches to the static com-DBLP graph (317,080 vertices, 1,049,866 edges). Each run has five cumulative batches; a batch changes a fixed fraction of $|E|$, split 60% insertions / 40% deletions. We use batch sizes of 2%, 5%, 10%, and 20% of $|E|$ - progressively more aggressive. We track the modularity of our incremental variants (ND, DF, DS) against the cuGraph and NetworKit baselines, which re-cluster the whole

graph after every batch. Figure 5.3 reports the result for the experiment for size batches of 2% ($x = 0$ is the initial static clustering). Table 5.24 reports the results for the larger batches. These are presented as a table because the lines in a line chart would be very close.

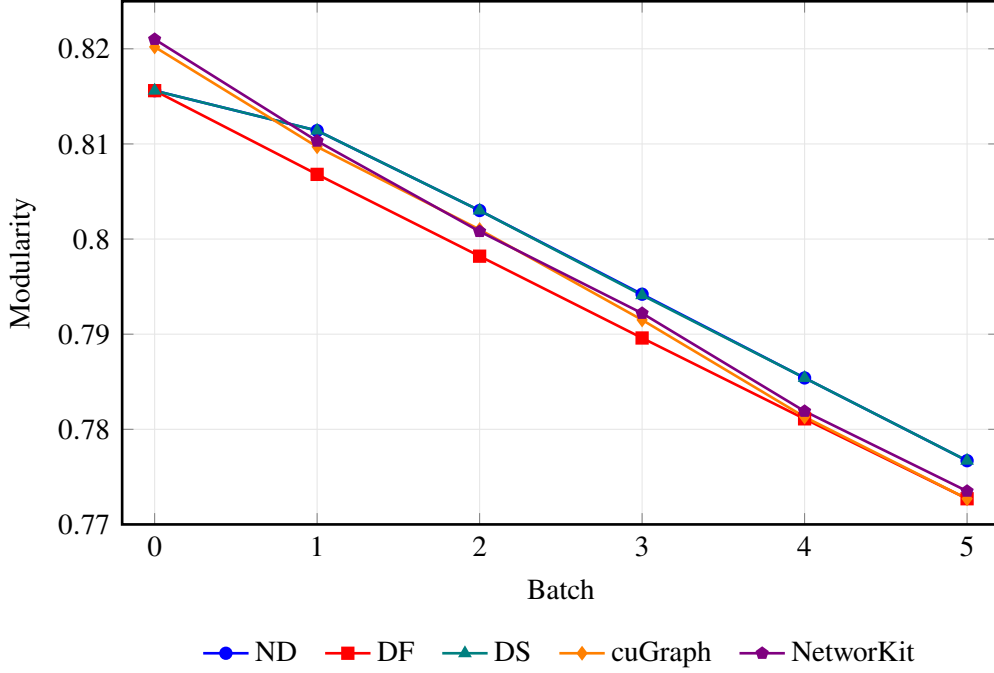


Figure 5.3: Modularity per batch on com-DBLP for a batch size of 2% of $|E|$. The ND and DS variants pretty much coincide.

Incremental updates do not degrade faster than re-clustering. The five curves stay within 0.01 of one another at every batch in Figure 5.3. In fact from Table 5.24 we see that the dynamic variants outperform cuGraph and NetworKit albeit marginally.

Degradation scales with batch size: Modularity falls monotonically for every method as batches accumulate, because the random rewiring steadily erodes the graph’s community structure.

Batch	ND	DF	DS	cuGraph	NetworKit
Batch size = 5% of E					
0	0.8162	0.8162	0.8162	0.8202	0.8222
1	0.7979	0.7944	0.7979	0.7958	0.7964
2	0.7764	0.7732	0.7764	0.7724	0.7742
3	0.7554	0.7527	0.7554	0.7509	0.7536
4	0.7353	0.7330	0.7353	0.7292	0.7314
5	0.7163	0.7143	0.7163	0.7105	0.7121
Batch size = 10% of E					
0	0.8190	0.8190	0.8190	0.8202	0.8207
1	0.7775	0.7751	0.7775	0.7715	0.7726
2	0.7362	0.7342	0.7362	0.7289	0.7316
3	0.6982	0.6967	0.6982	0.6896	0.6916
4	0.6631	0.6620	0.6631	0.6548	0.6578
5	0.6309	0.6300	0.6309	0.6245	0.6258
Batch size = 20% of E					
0	0.8199	0.8199	0.8199	0.8202	0.8208
1	0.7338	0.7327	0.7338	0.7271	0.7296
2	0.6596	0.6589	0.6596	0.6530	0.6548
3	0.5967	0.5962	0.5967	0.5909	0.5932
4	0.5432	0.5427	0.5432	0.5382	0.5413
5	0.4973	0.4969	0.4973	0.4936	0.4982

Table 5.24: Modularity per batch for the larger batch sizes (5%, 10%, 20% of $|E|$)

CHAPTER 6

CONCLUSION

In this work we implemented, from scratch in CUDA, GPU Louvain community detection consisting of a static algorithm and three dynamic variants - Naive-Dynamic (ND), Delta-Screening (DS), and Dynamic-Frontier (DF). The implementation builds on my UGRC work and resolves several bugs and correctness issues present in it.

For the dynamic setting, the three variants process each batch incrementally from the previous partition instead of re-clustering. Across the SNAP temporal graphs a batch update is several times faster than re-clustering with cuGraph or NetworKit and hundreds of times faster than NetworkX, with little to no reduction in quality. The performance of the static implementation on some graphs exceeds every baseline but crucially suffers in graphs with higher vertex degrees.

6.1 FUTURE WORK

6.1.1 Algorithmic Improvements

- **Not just a port.** Our dynamic variants largely follow the Delta-Screening and DF-Louvain papers. Modifications to the algorithm which are more suitable for the GPU can be studied.
- **Vertex updates.** The implementation currently supports edge insertions and deletions over a fixed vertex set. There is no explicit support for vertex addition and deletion.
- **Profiling.** Further profiling is always possible. A thorough profiling of the time taken by each kernel will allow us to optimize the code which is the bottleneck.
- **Optimisations.** The two known targets are the issues of load imbalance on high-degree graphs and memory efficiency, which currently limits the largest processable graphs. The former can be solved with degree adaptive assignment of threads. The latter will need careful redesign of memory allocation and data structures

used, batched processing as in [Chou and Ghosh \(2022\)](#) or a shared-memory based implementation.

- **Leiden.** The Leiden algorithm refines Louvain and provides more guarantees. Investigating or pivoting to a dynamic GPU Leiden is a promising direction, since the same warm-start and affected-set machinery should apply. Through the literature review I have found in multiple papers that Leiden outperforms Louvain.
- **More leeway in community structure change.** The current implementations of the dynamic algorithms do not support easy increase in the number of communities but instead move nodes around existing communities. When there are many deletions this can be a drawback, therefore we would like to extend the implementation to handle this.

6.1.2 Evaluation Improvements

- **Richer quality metrics.** We compare partitions only through modularity and the number of communities. The actual vertex-level structure of the partitions is never compared directly. Modularity may not be a perfect metric - it suffers from the resolution limit. Other measures such as modularity density which were mentioned in [Cordeiro et al. \(2016\)](#) can be used. We can also utilise other metrics such as NMI (Normalized Mutual Information), ARI(Adjusted Rand Inde) if ground truth communities exist.
- **More baseline algorithms.** We currently baseline only with cuGraph, NetworkX and NetworKit with re-runs per batch for the dynamic algorithm. Benchmarking directly against some dynamic implementations such as the the DF-Louvain OpenMP reference implementation [Sahu \(2024\)](#) which is state of the art on CPUs would be a good comparison. Additionally, we can benchmark against static implementations such as GVE-Louvain and ν -Louvain which are relatively recent and show good performance.
- **Robust timings.** Current reported times are from single runs. Averaging over repeated runs would make the measurements more reliable.
- **Larger graphs.** Graphs such as uk-2007 would not currently fit in memory (several billion edges). Benchmarking at this scale first requires some memory efficiency work. Currently, the G4 (RTX PRO 6000 Blackwell) supports 96GB while the H100 (Hopper-class) and A100 (Ampere-class) support 80GB on Google Colab.
- **Benchmark Graphs.**
 - We can benchmark on more temporal graphs on the dynamic algorithm side, survey whether there are any temporal graphs with deletions (could not find one easily).

- Expand evaluation on weighted graphs and weight-change updates.

6.1.3 Publication

If time permits, with the assistance of Prof. Rupesh Nasre (my guide), I consider consolidating the implementation and the evaluation into a publication. Candidate venues are parallel-computing journals such as IJPP, JPDC, and IEEE TPDS, or conferences such as next year's ICPP.

APPENDIX A

GENERAL NOTES

A.1 THE LOCAL-MOVING KERNEL

Listing A.1 gives the full CUDA source of the vertex-parallel local-moving kernel summarised (as pseudocode) in the Implementation section.

```
1  __global__ void localMove_kernel(  
2      int N, const int* off, const int* dst, const double* w,  
3      int* C, const double* K, double* Sigma,  
4      int* active, int* range, int isDF,  
5      const long long* htOff, int* htKey, double* htVal,  
6      double M, double* dQ_accum, int* commLock)  
7  {  
8      int v = blockIdx.x * blockDim.x + threadIdx.x;  
9      if (v >= N || active[v] == 0) return;  
10     if (range != nullptr && range[v] == 0) { active[v] = 0; return; }  
11     active[v] = 0; // consume this vertex  
12  
13     long long base = htOff[v];  
14     int cap = (int)(htOff[v + 1] - htOff[v]);  
15     // power of two  
16     int mask = cap - 1;  
17     for (int s = 0; s < cap; ++s) htKey[base + s] = -1;  
18     // clear slice  
19  
20     int d = C[v]; double ki = K[v];  
21  
22     // accumulate weight from v to each neighbouring community  
23     for (int e = off[v]; e < off[v + 1]; ++e) {  
24         int u = dst[e]; if (u == v) continue;  
25         int c = C[u]; double we = w[e];  
26         int slot = (int)(hash_u(c) & mask);  
27         while (true) {  
28             int k = htKey[base + slot];  
29             if (k == c) { htVal[base + slot] += we; break; }  
30             if (k == -1) { htKey[base + slot] = c; htVal[base + slot]  
31                 = we; break; }  
32             slot = (slot + 1) & mask;  
33         }  
34     }
```



```

34
35 // optimistic best target (Sigma may be stale)
36 double Kid = 0.0;
37 for (int slot = (int)(hash_u(d) & mask); ; slot = (slot + 1) &
    mask) {
38     int k = htKey[base + slot];
39     if (k == d) { Kid = htVal[base + slot]; break; }
40     if (k == -1) break;
41 }
42 double Sd = Sigma[d];
43 int best_c = d; double best_dQ = 0.0;
44 for (int s = 0; s < cap; ++s) {
45     int c = htKey[base + s];
46     if (c == -1 || c == d) continue;
47     double Kic = htVal[base + s], Sc = Sigma[c];
48     double dQ = 2.0 * (Kic - Kid) / M
49         - 2.0 * ki * (ki + Sc - Sd) / (M * M);
50     if (dQ > best_dQ + MOVE_EPS ||
51         (dQ > best_dQ - MOVE_EPS && c < best_c)) { best_dQ = dQ;
52         best_c = c; }
53 }
54 if (best_c == d || best_dQ <= MOVE_EPS) return;
55
56 // lock the (d, best_c) community pair -- ORDERED to avoid
57 // deadlock
58 int lo = min(d, best_c), hi = max(d, best_c);
59 while (atomicCAS(&commLock[lo], 0, 1) != 0) {}
60 while (atomicCAS(&commLock[hi], 0, 1) != 0) {}
61
62 // re-scan with frozen membership -> EXACT gain, then commit
63 double Kid2 = 0.0, Kic2 = 0.0;
64 for (int e = off[v]; e < off[v + 1]; ++e) {
65     int cu = C[dst[e]]; double we = w[e];
66     if (cu == d) Kid2 += we;
67     else if (cu == best_c) Kic2 += we;
68 }
69 double dQ2 = 2.0 * (Kic2 - Kid2) / M
70     - 2.0 * ki * (ki + Sigma[best_c] - Sigma[d]) / (M * M)
71     ;
72 if (dQ2 > MOVE_EPS) {
73     Sigma[d] -= ki; Sigma[best_c] += ki; // only the lock
74     // holders touch these
75     C[v] = best_c;
76     atomicAdd(dQ_accum, dQ2);
77     __threadfence();
78     for (int e = off[v]; e < off[v + 1]; ++e) {
79         active[dst[e]] = 1; // re-activate
80     }
81 }

```

```

76         neighbours
           if (isDF && range != nullptr) range[dst[e]] = 1; // DF:
           grow the frontier
77     }
78 }
79 __threadfence();
80 atomicExch(&commLock[hi], 0);
81 atomicExch(&commLock[lo], 0); // release in
    reverse order
82 }

```

Listing A.1: Vertex-parallel local move.

REFERENCES

1. **Aynaud, T.** and **J.-L. Guillaume**, Static community detection algorithms for evolving networks. In *Proceedings of the 8th International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOpt'10)*. Avignon, France, 2010. HAL: inria-00492058.
2. **Bhowmick, A., S. Vadhiyar,** and **V. PV** (2022). Scalable multi-node multi-gpu louvain community detection algorithm for heterogeneous architectures. *Concurrency and Computation: Practice and Experience*, **34**(17), e6987. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/cpe.6987>.
3. **Bhowmik, A.** and **S. Vadhiyar**, Hydetect: A hybrid cpu-gpu algorithm for community detection. In *2019 IEEE 26th International Conference on High Performance Computing, Data, and Analytics (HiPC)*. 2019.
4. **Blondel, V. D., J.-L. Guillaume, R. Lambiotte,** and **E. Lefebvre** (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, **2008**(10), P10008.
5. **Brandes, U., D. Delling, M. Gaertler, R. Görke, M. Hoefer, Z. Nikoloski,** and **D. Wagner** (2008). On modularity clustering. *IEEE Transactions on Knowledge and Data Engineering*, **20**(2), 172–188.
6. **Cheong, C. Y., H. P. Huynh, D. Lo,** and **R. S. M. Goh**, Hierarchical parallel algorithm for modularity-based community detection using GPUs. In *Euro-Par 2013 Parallel Processing*, volume 8097 of *Lecture Notes in Computer Science*. Springer, 2013.
7. **Chou, H.-Y.** and **S. Ghosh**, Batched graph community detection on GPUs. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques (PACT '22)*. ACM, Chicago, IL, USA, 2022. Software: <https://github.com/sg0/nido>.
8. **Clauset, A., M. Newman,** and **C. Moore** (2005). Finding community structure in very large networks. *Physical review. E, Statistical, nonlinear, and soft matter physics*, **70**, 066111.
9. **Cordeiro, M., R. Sarmiento,** and **J. Gama** (2016). Dynamic community detection in evolving networks using locality modularity optimization. *Social Network Analysis and Mining*, **6**.
10. **Fazlali, M., E. Moradi,** and **H. T. Malazi** (2017). Adaptive parallel louvain community detection on a multicore platform. *Microprocessors and Microsystems*, **54**, 26–34.
11. **Fortunato, S.** and **M. Barthélemy** (2007). Resolution limit in community detection.

12. **Gawande, N., S. Ghosh, M. Halappanavar, A. Tumeo, and A. Kalyanaraman** (2022). Towards scaling community detection on distributed-memory heterogeneous systems. *Parallel Computing*, **111**, 102898. Open access (CC BY-NC-ND 4.0). Paper presents cuVite, a distributed multi-GPU Louvain library.
13. **Ghosh, S., M. Halappanavar, A. Tumeo, A. Kalyanaraman, and A. H. Gebremedhin**, Scalable distributed memory community detection using Vite. *In Proc. 22nd IEEE High Performance Extreme Computing Conf. (HPEC)*. IEEE, 2018a.
14. **Ghosh, S., M. Halappanavar, A. Tumeo, A. Kalyanaraman, H. Lu, D. Chavarria-Miranda, A. Khan, and A. H. Gebremedhin**, Distributed louvain algorithm for graph community detection. *In Proc. 32nd IEEE Int. Parallel and Distributed Processing Symp. (IPDPS)*. IEEE, 2018b.
15. **Leskovec, J. and A. Krevl** (2014). SNAP Datasets: Stanford large network dataset collection. <http://snap.stanford.edu/data>.
16. **Lu, H., M. Halappanavar, and A. Kalyanaraman** (2015). Parallel heuristics for scalable community detection. *Parallel Computing*, **47**, 19–37.
17. **Mohammadi, M., M. Fazlali, and M. Hosseinzadeh** (2021). Accelerating Louvain community detection algorithm on graphic processing unit. *The Journal of Supercomputing*, **77**(6), 6056–6077.
18. **Moradi, E., M. Fazlali, and H. Tabatabaee Malazi**, Fast parallel community detection algorithm based on modularity. 2015.
19. **Muthavarapu, H.** (). Starplat dynamic louvain code. https://github.com/gajendra-iitm/starplat/tree/main/graphcode/dynamic_louvain_community_detection.
20. **Naim, M., F. Manne, M. Halappanavar, and A. Tumeo**, Community detection on the GPU. *In 2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 2017.
21. **Nasre, R.** (). Professor rupesh nasre courses - cs6023. <https://www.cse.iitm.ac.in/~rupesh/teaching/gpu/jan25/>.
22. **Newman, M. E. J. and M. Girvan** (2004). Finding and evaluating community structure in networks. *Physical Review E*, **69**(2), 026113.
23. **NVIDIA Corporation** (). Rapids cudagraph: Gpu graph analytics library. <https://github.com/rapidsai/cugraph>.
24. **NVIDIA Corporation** (2026). *NVIDIA CUDA Compiler Driver*

- NVCC. NVIDIA Corporation. URL <https://docs.nvidia.com/cuda/cuda-compiler-driver-nvcc/>.
25. **Raghavan, U. N., R. Albert, and S. Kumara** (2007). Near linear time algorithm to detect community structures in large-scale networks. *Physical Review E*, **76**(3), 036106.
 26. **Rotta, R. and A. Noack** (2011). Multilevel local search algorithms for modularity clustering. *ACM Journal of Experimental Algorithmics*, **16**.
 27. **Sahu, S.** (2023). GVE-Louvain: Fast louvain algorithm for community detection in shared memory setting. *arXiv preprint arXiv:2312.04876*. URL <https://arxiv.org/abs/2312.04876>.
 28. **Sahu, S.** (2024). Df louvain: Fast incrementally expanding approach for community detection on dynamic graphs. URL <https://arxiv.org/abs/2404.19634>.
 29. **Sahu, S.** (2025). CPU vs. GPU for community detection: Performance insights from GVE-Louvain and ν -Louvain. *arXiv preprint arXiv:2501.19004*. URL <https://arxiv.org/abs/2501.19004>. Software: <https://github.com/puzzlef/louvain-communities-cuda> (ν -Louvain) and <https://github.com/puzzlef/louvain-communities-openmp> (GVE-Louvain).
 30. **Seifikar, M., S. Farzi, and M. Barati** (2020). C-blondel: An efficient louvain-based dynamic community detection algorithm. *IEEE Transactions on Computational Social Systems*, **7**(2), 308–318.
 31. **Shang, J., L. Liu, F. Xie, Z. Chen, J. Miao, X. Fang, and C. Wu** (2014). A real-time detecting algorithm for tracking community structure of dynamic networks. *CoRR*, **abs/1407.2683**. URL <http://arxiv.org/abs/1407.2683>.
 32. **Staudt, C. L. and H. Meyerhenke** (2016). Engineering parallel algorithms for community detection in massive networks. *IEEE Transactions on Parallel and Distributed Systems*, **27**(1), 171–184.
 33. **Traag, V. A., L. Waltman, and N. J. van Eck** (2019). From louvain to leiden: guaranteeing well-connected communities. *Scientific Reports*, **9**(1), 5233.
 34. **Zarayeneh, N. and A. Kalyanaraman** (2021). Delta-screening: A fast and efficient technique to update communities in dynamic graphs. *IEEE Transactions on Network Science and Engineering*, **PP**, 1–1.